爬虫与反爬虫

面试 100 题 · 系统化详解

从 HTTP 基础到十亿级 URL Frontier，从 Scrapy 到 Browser Pool，从数据质量到 Bot Management

题目驱动 · 原理推导 · 工程实践 · 面试追问 · 系统设计

2026 专业版

前言：这本题库解决什么问题

很多“爬虫面试题”停留在 requests、XPath、代理池等 API 层面，但真实中高级岗位更关心：你是否理解 HTTP 语义、异步调度、动态页面、分布式 URL Frontier、增量抓取、数据质量、可观测性，以及站点侧的限流和 Bot Management。

本手册采用“题目驱动式”技术面试辅导结构：每题给出考察点、推荐回答、深度解析、易错点和追问。结构上借鉴经典技术面试辅导书的逐题剖析方法，但内容完全围绕现代 Web 数据采集与防护工程重新组织，不复刻任何受版权保护的原文。

阅读边界

- 反爬相关内容主要从网站防护、风险检测和系统设计角度展开。
- 对验证码、挑战、访问控制等机制，重点讨论识别、治理与合法替代路径，不提供绕过站点安全边界的实操教程。
- 实际采集应同时考虑 robots 协议、站点条款、授权、隐私、数据许可和所在地法律要求。

如何使用本书

目标	优先章节	建议做法
初级 Python/数据采集岗位	1-4、6	先能完整解释 HTTP/DOM/Scrapy，再手写小型 crawler。
中级爬虫工程师	1-6、9	重点掌握异步、Browser 分层、去重、增量与数据质量。
高级/平台岗位	5-10	按容量估算、状态平面、调度约束和故障恢复回答系统设计。
网站安全/反自动化岗位	7-10	重点理解风险评分、限流、误报、Feature Pipeline 与策略治理。

建议每道题先用 60-90 秒口述“推荐回答”，再用 3-5 分钟回答追问。系统设计题要求先估量级、再画状态与数据流，最后讨论一致性、成本、SLO 和失败模式。

全书知识地图

章节	题号	核心能力
第一章 HTTP、URL、Robots 与缓存	Q001-Q010	从网络语义出发理解爬虫。重点不是记

章节	题号	核心能力
		API，而是理解请求语义、状态管理、缓存、重定向与站点访问策略。
第二章 HTML、DOM 与数据抽取	Q011-Q020	从“能选到元素”升级为“稳定、可验证、可演进的数据抽取系统”。
第三章 Scrapy、异步与并发	Q021-Q030	理解 Scrapy 的事件驱动架构、调度、限速与 I/O/CPU 边界。
第四章 JavaScript、SPA 与浏览器自动化	Q031-Q040	掌握动态渲染、网络事件、浏览器会话隔离和成本分层。
第五章 分布式爬虫	Q041-Q050	把单机脚本升级为可扩展、可恢复、可治理的 URL Frontier 与工作流系统。
第六章 去重、调度、存储与数据质量	Q051-Q060	决定抓取结果能否长期复用、增量更新和稳定进入下游。
第七章 反爬检测原理	Q061-Q070	从防守视角理解现代 Bot Detection 的多信号、风险评分与误报成本。
第八章 服务端防爬体系	Q071-Q080	限流、WAF、Bot Management、挑战策略与模型评估。
第九章 稳定性、可观测性与数据质量	Q081-Q090	生产爬虫最重要的不是“能跑”，而是“能发现自己已经错了”。
第十章 高级系统设计	Q091-Q100	用容量估算、约束建模和分层架构回答 10^8 到 10^9 URL 级系统设计题。

目录（按题号）

第一章 HTTP、URL、Robots 与缓存

Q001 浏览器访问一个 URL 到拿到 HTML，中间到底发生了什么?

Q002 GET 和 POST 在爬虫里的核心区别是什么?

Q003 什么是幂等性？为什么爬虫工程必须理解?

Q004 301、302、307、308 有什么区别?

Q005 304 Not Modified 对增量爬虫有什么意义?

Q006 Cookie、Session、Token 三者是什么关系?

Q007 robots.txt 到底是什么？是不是访问授权文件?

Q008 robots.txt 的 Allow 与 Disallow 冲突怎么办?

Q009 Sitemap 与 URL Frontier 有什么关系?

Q010 什么是 polite crawling?

第二章 HTML、DOM 与数据抽取

Q011 XPath 和 CSS Selector 怎么选?

Q012 为什么绝对 XPath 很脆弱?

Q013 DOM Tree 与原始 HTML 为什么可能不同?

Q014 正文抽取为什么比 XPath 难得多?

Q015 页面结构变化后，如何避免大规模产出空数据?

Q016 JSON-LD 为什么对 crawler 很重要?

Q017 页面有多个价格，到底应该抓哪个?

Q018 如何处理分页?

Q019 无限滚动页面为什么难抓?

Q020 HTML parser 应该选择严格还是容错?

第三章 Scrapy、异步与并发

Q021 画出 Scrapy 的架构。

Q022 Scrapy 为什么通常比 requests + ThreadPool 更适合高并发抓取?

Q023 Reactor / Event Loop 是什么?

Q024 Scrapy Scheduler 是做什么的?

Q025 Downloader Middleware 和 Spider Middleware 有什么区别?

Q026 Item Pipeline 为什么不能和 Spider 混在一起?

Q027 Scrapy 如何控制并发?

Q028 AutoThrottle 是什么?

Q029 为什么不能在 Scrapy callback 里做长时间 CPU 运算?

Q030 Scrapy 如何实现 graceful shutdown?

第四章 JavaScript、SPA 与浏览器自动化

Q031 requests 能拿到 HTML，为什么还是没有数据?

Q032 SPA 爬虫应该先分析 DOM 还是 Network?

Q033 Playwright 的 Browser、Context、Page 分别是什么?

Q034 Selenium 和 Playwright 的主要区别?

Q035 为什么 sleep(5) 是浏览器自动化的坏味道?

Q036 什么是 explicit wait?

Q037 为什么不建议混用 implicit wait 和 explicit wait?

Q038 什么是 Chrome DevTools Protocol（CDP）?

Q039 Playwright 如何观察 Network Request?

Q040 Browser Crawler 最大的性能问题是什么?

第五章 分布式爬虫

Q041 单机爬虫怎么演化成分布式爬虫?

Q042 分布式 crawler 最难的问题是什么?

Q043 URL Frontier 是什么?

Q044 URL 如何分片到不同 Worker?

Q045 如何解决热门域名导致的 Partition Skew?

Q046 Kafka 能不能直接当 URL Frontier?

Q047 Redis 在分布式 crawler 中适合做什么?

Q048 如何保证 Worker Crash 后任务不丢?

Q049 为什么 Exactly-once Crawling 几乎没有必要?

Q050 什么是 Backpressure?

第六章 去重、调度、存储与数据质量

Q051 URL 去重最简单怎么做?

Q052 Bloom Filter 为什么适合 URL 去重?

Q053 Bloom Filter 假阳性率怎么估?

Q054 URL Canonicalization 为什么困难?

Q055 URL 去重和内容去重有什么区别?

Q056 网页内容如何做近重复检测?

Q057 什么时候使用 SimHash?

Q058 数据应该存 MySQL、MongoDB 还是对象存储?

Q059 为什么原始 HTML 最好保留?

Q060 成熟 crawler 的数据模型应该有哪些 Metadata?

第七章 反爬检测原理

Q061 服务端如何判断一个请求是不是 Bot?

Q062 为什么只改 User-Agent 不能解决 Bot Detection?

Q063 什么是浏览器指纹?

Q064 TLS Fingerprint 是什么?

Q065 IP Reputation 是怎么工作的?

Q066 为什么数据中心 IP 与普通用户网络的风险画像可能不同?

Q067 行为检测主要看什么?

Q068 为什么固定频率访问容易形成异常特征?

Q069 CAPTCHA 在 Bot Management 体系中处于什么位置?

Q070 为什么“出现 CAPTCHA 就自动破解”不是成熟 crawler 的设计?

第八章 服务端防爬体系

Q071 如何设计最简单的 Rate Limiter?

Q072 Sliding Window 有什么优势?

Q073 Token Bucket 原理是什么?

Q074 Leaky Bucket 和 Token Bucket 区别?

Q075 只按 IP Rate Limit 有什么问题?

Q076 如何保护一个昂贵 Search API?

Q077 WAF 与 Bot Management 有什么区别?

Q078 Challenge、Block、Throttle 应该如何选择?

Q079 如何评价一个反爬模型?

Q080 如何设计反爬系统的 Feature Pipeline?

第九章 稳定性、可观测性与数据质量

Q081 Crawler 最重要的指标是什么?

Q082 为什么 HTTP 200 不能代表抓取成功?

Q083 如何发现网站改版?

Q084 什么叫 Silent Data Corruption?

Q085 如何做 Crawler Regression Test?

Q086 如何做 Canary Crawling?

Q087 Retry 应如何设计?

Q088 哪些错误不应该反复 Retry?

Q089 Dead Letter Queue 是什么?

Q090 如何定位“爬虫今天突然变慢”?

第十章 高级系统设计

Q091 设计一个每天抓取 1 亿网页的系统。

Q092 设计一个搜索引擎 Web Crawler。

Q093 如何设计 Recrawl Scheduler?

Q094 如何检测 Crawl Trap?

Q095 如何抓取 1000 个域名，同时保证每个域名不超过自己的 QPS?

Q096 设计一个“HTTP + Browser”混合爬虫。

Q097 Browser Pool 应该怎么设计?

Q098 设计一个企业级反爬系统。

Q099 Crawler 成功率从 99% 突然降到 30%，怎么排查?

Q100 终极题：设计一个十亿级 URL、可增量更新、合规、稳定且不压垮目标网站的爬虫平台。

第一章 HTTP、URL、Robots 与缓存

从网络语义出发理解爬虫。重点不是记 API，而是理解请求语义、状态管理、缓存、重定向与站点访问策略。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q001 浏览器访问一个 URL 到拿到 HTML，中间到底发生了什么？ ★★☆☆☆

考察点

- URL 解析与 DNS
- TCP/QUIC、TLS、HTTP
- CDN/WAF/反向代理与应用链路

推荐回答

不要把答案缩成“发 HTTP 请求”。一个完整抓取链路至少包含 URL 解析、DNS、连接建立、TLS、HTTP 请求、边缘代理/CDN/WAF、源站处理、响应解压与 HTML 解析。

如果是 HTTP/2 或 HTTP/3，还应理解多路复用、连接复用以及传输协议变化并不会取消应用层的并发和限速约束。

```
URL → DNS → TCP/QUIC → TLS → HTTP → CDN/WAF → Origin
                                        ↓
                                  Response → Parse
```

深度解析

工程排障时要把总耗时拆成 DNS、连接、TLS、TTFB、下载、解析和存储；否则“慢”无法定位。

爬虫吞吐提升通常来自减少等待、提高连接复用和并发利用率，而不是盲目增加线程。

易错点

- 只谈 requests.get()；忽略 DNS/TLS/重定向/压缩。
- 把 HTTP/2 多路复用误解为“服务器可以无限并发”。

面试追问

1. HTTP/2 下为什么仍要 per-host concurrency?

2. TTFB 突然上涨时你先看哪三类指标?

一句话记忆

- URL 不是直接变成 HTML，而是穿过一条可观测、可限速、会失败的网络与应用链路。

Q002 GET 和 POST 在爬虫里的核心区别是什么? ★☆☆☆☆

考察点

- HTTP 方法语义
- 安全性与幂等性
- 自动化访问的副作用边界

推荐回答

GET 通常用于读取资源，并被定义为 safe；POST 通常表示提交处理，不保证幂等。面试中不要只说“GET 参数在 URL、POST 参数在 Body”，这不是最重要的区别。

爬虫系统应区分内容发现请求与可能造成副作用的业务请求。遇到表单、点赞、下单、删除等动作时，不能因为接口可调用就自动执行。

深度解析

重试策略必须和方法语义绑定：超时后重发一个幂等读取请求通常可以接受；对非幂等写操作则需要幂等键、业务确认或根本不自动重试。

易错点

- 认为 POST 一定更安全或更隐蔽。
- 把“可以发请求”误当成“可以自动化执行该业务动作”。

面试追问

1. PUT 为什么通常被视为幂等，而 POST 不一定?
2. 302 重定向后客户端对 POST 可能发生什么?

一句话记忆

- 方法差异首先是语义与副作用，其次才是参数放在哪里。

Q003 什么是幂等性？为什么爬虫工程必须理解? ★★☆☆☆

考察点

- 幂等操作
- 重试语义
- 至少一次执行与副作用

推荐回答

幂等可理解为重复执行同一操作，不应产生额外业务副作用。爬虫或采集平台中，超时、网络抖动、Worker 崩溃和队列重投都会导致同一个任务被重复执行。

因此工程上通常采用 at-least-once delivery，再通过幂等存储、内容哈希、唯一键或 request id 消除重复结果。

深度解析

“HTTP 请求是否幂等”与“数据处理是否幂等”要分开。例如 GET 获取网页通常幂等，但将解析结果 append 到无唯一键日志表并不是幂等。

一个成熟管线应让 Fetch、Parse、Store 各阶段分别定义自己的幂等键。

易错点

- 把幂等理解为“响应内容永远相同”。
- 只在请求层考虑幂等，不在存储层考虑。

面试追问

1. 如何给 item 写入设计幂等键?
2. 消息消费至少一次时如何防止重复入库?

一句话记忆

- 分布式系统里，重复执行是常态；幂等性是把重复执行变成可接受的关键。

Q004 301、302、307、308 有什么区别? ★★☆☆☆

考察点

- 永久/临时重定向
- 方法是否保留
- canonical 与去重

推荐回答

301/308 通常表达永久迁移，302/307 通常表达临时迁移。307/308 明确保留原请求方法和请求体；301/302 在历史客户端中存在将 POST 转为 GET 的兼容行为。

爬虫应保存 original_url、redirect_chain 与 final_url，避免把重定向链丢掉，否则 canonicalization、来源追踪和增量抓取都会变得困难。

深度解析

永久重定向可作为 canonical 信号之一，但不应立刻删除原始来源记录。对反复变化的 302，应保留观察窗口。

易错点

- 只记住“301 永久、302 临时”，不谈 307/308 的方法语义。

面试追问

1. 如果 A 301 到 B，B 又 canonical 到 C，去重键怎么设计?
2. 跨域重定向时 Cookie 与 Authorization 应怎么处理?

一句话记忆

- 重定向不仅改变 URL，还可能改变请求方法、缓存策略和 canonical 关系。

Q005 304 Not Modified 对增量爬虫有什么意义? ★★★☆☆

考察点

- ETag / Last-Modified
- 条件请求
- 增量抓取与带宽优化

推荐回答

第一次抓取后保存 ETag 和 Last-Modified；下次发送 If-None-Match 或 If-Modified-Since。若内容未变化，服务器可返回 304，客户端复用本地实体。

它把“全量重新下载”优化成“用低成本条件请求确认是否变化”，对高频 recrawl、静态文章和大正文页面非常有价值。

深度解析

ETag 可能是弱验证器，也可能因负载均衡或动态模板变化而不稳定，因此还应保留内容哈希作最终判定。

304 只减少响应实体传输，不等于不产生服务器成本；调度仍应遵循站点访问预算。

易错点

- 把 304 当错误。
- 只保存 Last-Modified，不考虑 ETag 与内容哈希。

面试追问

1. 弱 ETag 与强 ETag 有什么区别?
2. 如果服务器从不返回缓存验证器，如何实现增量判断?

一句话记忆

- 增量爬虫的核心不是“再次抓”，而是“以最低成本判断是否值得再次处理”。

Q006 Cookie、Session、Token 三者是什么关系? ★★☆☆☆

考察点

- 客户端状态
- 服务端会话
- 认证与授权边界

推荐回答

Cookie 是客户端保存并按作用域随请求携带的数据；Session 常指服务器端会话状态，Cookie 中可能只保存 session_id；Token 则可能本身携带可验证的身份或授权信息。

自动化客户端必须按身份隔离 CookieJar、localStorage、CSRF 状态和 Token，不能为了省事让多个身份共享一个状态容器。

深度解析

浏览器自动化中，BrowserContext 是天然的会话隔离边界；HTTP 客户端中则需要独立 Session/CookieJar。状态污染会造成串号、AB 实验错乱或错误权限。

易错点

- 认为 Cookie 就等于 Session。
- 把登录状态复制到不相关任务而忽略权限与合规边界。

面试追问

1. SameSite 对自动化请求有什么影响?
2. CSRF token 为什么经常和 Session 绑定?

一句话记忆

- 身份状态要隔离；Cookie 是载体，Session/Token 是更高层的身份与授权机制。

Q007 robots.txt 到底是什么？是不是访问授权文件？ ★★☆☆☆

考察点

- RFC 9309
- 爬虫协商
- 认证/授权与 robots 的边界

推荐回答

robots.txt 是 Robots Exclusion Protocol，用于站点向自动化爬虫表达哪些 URI 应被访问或排除。它是协议层的爬虫协商机制，不是认证或授权系统。

专业 crawler 应在 host 首次进入 frontier 时获取并缓存 robots policy，把策略检查置于请求调度之前，同时记录策略版本、获取时间与失败行为。

深度解析

robots.txt 与法律许可、合同条款、登录授权并不等价。企业采集系统应把 robots policy、业务授权、数据许可和站点条款拆成不同治理信号。

易错点

- 把“robots 允许”理解成“法律上当然允许”。
- 把“robots 不可达”默认解释成无限制访问。

面试追问

1. robots.txt 返回 5xx 时如何处理?
2. 为什么 robots parser 的缓存也要有 TTL?

一句话记忆

- robots.txt 是 crawler 协商，不是权限系统。

Q008 robots.txt 的 Allow 与 Disallow 冲突怎么办？ ★★★☆☆

考察点

- 最长匹配原则
- User-agent group
- parser 一致性

推荐回答

RFC 9309 的核心是基于匹配规则长度判断最具体的规则。不能简单做“看到 Disallow 就禁止”，也不能自己用几个 startswith 拼一个不兼容解析器。

生产系统应使用经过规范验证的 parser，缓存针对当前 User-Agent 的有效 group，并为规则解析与调度策略写单元测试。

深度解析

不同历史实现对 wildcard、百分号编码与大小写等细节曾有差异，因此面试中应强调“遵循规范且使用成熟 parser”，而非记一套私有语法。

易错点

- 自己写正则解析导致与标准不一致。

面试追问

1. 同长度 Allow 与 Disallow 如何处理?
2. URL 百分号编码在匹配前应该怎样处理?

一句话记忆

- robots 匹配是协议实现问题，最重要的是兼容性和可审计性。

Q009 Sitemap 与 URL Frontier 有什么关系? ★★☆☆☆

考察点

- URL discovery
- hint 与 truth 的区别
- 深层/孤立页面发现

推荐回答

Sitemap 是 URL 发现入口，可直接向 frontier 注入页面列表，减少单纯依赖链接遍历造成的遗漏，尤其适合深层页面、孤立页面和大站点。

但 Sitemap 只是 hint：其中 URL 可能重定向、404、过期或 canonical 到别处。Crawler 仍要经过 URL 规范化、策略校验、去重和优先级调度。

深度解析

可以把 Sitemap 的 lastmod 与历史 change rate 合并，用于 recrawl priority；但 lastmod 由站点声明，不应无条件信任。

易错点

- 把 Sitemap 当作最终数据库清单。

面试追问

1. Sitemap index 很大时如何流式处理?
2. lastmod 不可信时如何判断 freshness?

一句话记忆

- Sitemap 是发现加速器，不是事实数据库。

Q010 什么是 polite crawling? ★★☆☆☆

考察点

- per-host 并发
- 自适应限速
- Retry-After / backoff

推荐回答

Polite crawling 的目标是最大化有用吞吐，同时把对目标站点造成的负载控制在合理范围。常见机制包括 per-domain concurrency、download delay、自动节流、Retry-After、指数退避与请求预算。

关键是限速应围绕目标站点或服务资源，而不是只设置一个全局 QPS。

深度解析

固定 sleep 往往既浪费性能又不能保护站点：站点快时过慢、站点慢时仍可能过载。更成熟的策略会根据延迟、错误率和历史容量动态调节。

易错点

- 以“抓得越快越专业”为目标。
- 只有全局限速，没有 host 维度。

面试追问

1. 如何用 latency 调整目标并发?
2. 429 和 503 的退避策略应一样吗?

一句话记忆

- 好的 crawler 追求可持续吞吐，而不是瞬时最大 QPS。

第二章 HTML、DOM 与数据抽取

从“能选到元素”升级为“稳定、可验证、可演进的数据抽取系统”。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q011 XPath 和 CSS Selector 怎么选? ★☆☆☆☆

考察点

- 选择器表达能力
- 可读性
- 稳定性

推荐回答

CSS Selector 简洁、可读，适合 class/id/属性和常见层级关系；XPath 在祖先、兄弟、文本条件和复杂结构关系上表达能力更强。

选型原则不是“谁更强”，而是选择最简单、最稳定、最容易维护的表达。

深度解析

稳定 selector 应优先语义属性、稳定 id、data-*、结构化元数据；尽量避免依赖深层级和位置索引。

易错点

- 为了炫技写非常长的 XPath。
- 依赖 nth-child/绝对层级导致页面小改即失效。

面试追问

1. 如何给 selector 设计回归测试?
2. 文本匹配为什么容易受国际化影响?

一句话记忆

- 选择器的第一目标是长期稳定，不是语法复杂。

Q012 为什么绝对 XPath 很脆弱? ★★☆☆☆

考察点

- DOM 结构漂移
- 位置依赖
- 语义 selector

推荐回答

绝对 XPath 把 DOM 的每一层位置都写死，例如 /html/body/div[2]/div[4]/h2。页面只要插入一个 banner，后续索引全部偏移。

更可靠的定位应利用语义属性或局部关系，如 article、data-testid、schema.org、稳定 class/id，再从局部节点相对查找。

深度解析

长期维护中应对 selector 计算“脆弱度”：位置索引越多、层级越深、依赖展示类名越多，风险通常越高。

易错点

- 把页面截图看起来不变等同于 DOM 不变。

面试追问

1. 如果站点 class 名经过哈希构建，如何选稳定特征?
2. 你会怎么做多 selector fallback?

一句话记忆

- 定位应依赖语义，不依赖页面偶然的层级位置。

Q013 DOM Tree 与原始 HTML 为什么可能不同? ★★☆☆☆

考察点

- HTML 容错解析
- JavaScript 动态修改
- Browser DOM 与 server HTML

推荐回答

浏览器会修复非法 HTML、补全隐式节点，并在 JavaScript 运行后继续修改 DOM；template、iframe、shadow DOM 还会引入额外树结构。

因此“requests 拿到的 HTML”“HTML parser 生成的树”“浏览器最终 DOM”是三个不同层次。调试时必须明确自己观察的是哪一层。

深度解析

很多 selector 问题不是规则错，而是数据尚未进入初始 HTML。先判断内容来源，再决定是否需要浏览器。

易错点

- 把 DevTools Elements 面板里的 DOM 当成服务器原始响应。

面试追问

1. 如何确认一个节点是服务端渲染还是 JS 注入?
2. Shadow DOM 对普通 XPath 有什么影响?

一句话记忆

- 抓取动态网页前，先区分 server HTML、parser tree 和 runtime DOM。

Q014 正文抽取为什么比 XPath 难得多? ★★★☆☆

考察点

- 正文/模板分类
- 文本密度与链接密度
- 通用抽取算法

推荐回答

页面里同时存在导航、正文、广告、推荐、评论和页脚。真正问题不是找到 p 标签，而是判断每个 block 是否属于 main content。

通用抽取器会结合文本密度、链接密度、DOM 深度、标签类型、标点、段落长度和模板频率等信号，把问题建模为 block classification。

深度解析

新闻、论坛、文档站、代码页面的“正文”定义不同，因此通用抽取器仍需域级评测。生产系统应保留 raw HTML，支持离线重新抽取。

易错点

- 把所有 p 拼起来当正文。
- 只看正文非空，不评估 boilerplate 污染率。

面试追问

1. 如何衡量正文抽取质量?

2. 为什么模板污染会影响内容去重?

一句话记忆

- 正文抽取是页面语义分类问题，不是标签遍历问题。

Q015 页面结构变化后，如何避免大规模产出空数据? ★★★☆☆

考察点

- Schema validation
- 数据漂移报警
- 隔离与回滚

推荐回答

最危险的失败不是程序崩溃，而是 selector 返回空字符串后仍正常写库。正确系统必须在 extraction 后进行 schema validation，例如必填字段、长度、格式、值域和交叉一致性。

验证失败的数据进入 quarantine/DLQ，并触发字段缺失率、items/page、DOM signature 等指标报警。

深度解析

对高价值站点可采用 canary parser：新版本只处理少量页面，与旧版本做字段分布和成功率对比后再全量。

易错点

- 把空字符串当合法默认值。
- 只监控 HTTP 200，不监控字段质量。

面试追问

1. 如何区分“页面真的没有价格”和“selector 失效”?
2. 哪些字段适合强校验，哪些适合弱校验?

一句话记忆

- 生产爬虫必须能发现“程序没报错但数据已经坏了”。

Q016 JSON-LD 为什么对 crawler 很重要? ★★☆☆☆

考察点

- 结构化元数据
- schema.org

- 多源融合

推荐回答

很多页面通过 application/ld+json 暴露 Article、Product、Breadcrumb、Organization 等结构化信息。相比展示 DOM，它往往更稳定，也更接近业务语义。

工程上应优先尝试 JSON-LD、OpenGraph、Microdata 等结构化来源，再用 DOM 补齐缺失字段，并记录每个字段的来源。

深度解析

结构化元数据也可能过期、缺字段或与页面展示不一致，因此不能无条件信任。可做字段级 source priority 和一致性检查。

易错点

- 认为 JSON-LD 一定正确。
- 混合多个 Product 节点时没有实体对齐。

面试追问

1. 如何选择页面中多个 JSON-LD entity?
2. JSON-LD 与 DOM 价格不一致时保留哪个?

一句话记忆

- 结构化数据是高价值信号，但仍需要验证和实体对齐。

Q017 页面有多个价格，到底应该抓哪个？ ★★★☆☆

考察点

- 语义建模
- 业务 schema
- 时间与条件

推荐回答

“price”不是一个字符串，而是一组带条件的业务事实。至少要区分 list_price、sale_price、member_price、currency、promotion、availability 和 observation_time。

面试中如果只回答“选红色字体的价格”，说明还停留在 DOM 层；高级回答应把页面展示映射成领域模型。

深度解析

价格还可能受地区、登录状态、规格 SKU、优惠券和税费影响，因此原始抓取上下文也应作为 provenance 保存。

易错点

- 用单字段 price 覆盖所有价格。
- 忽略 SKU/地区/会员状态导致跨页面错误比较。

面试追问

1. 如何设计商品价格历史表?
2. 同一商品多个 SKU 怎么做实体主键?

一句话记忆

- 稳定采集的终点是结构化业务事实，而不是字符串。

Q018 如何处理分页? ★★☆☆☆

考察点

- 页码/next/cursor
- 停止条件
- 循环检测

推荐回答

分页可能是 page 参数、显式 next 链接，也可能是 cursor/token。应优先从响应中提取明确的 next relation，而不是假设 page += 1 永远成立。

必须设置最大页数、空页停止、重复页停止、cursor 重复检测与 URL cycle detection，防止 crawl trap。

深度解析

cursor-based API 常把状态编码在不可解释 token 中；不要自行构造，应该沿服务器返回的合法 next cursor 前进。

易错点

- 无限 page++。
- 只根据 HTTP 200 判断还有下一页。

面试追问

1. 如果排序变化导致分页重复/漏项怎么办?
2. 如何在增量抓取中稳定遍历分页列表?

一句话记忆

- 分页的关键是遵循服务端给出的“下一步状态”，并定义可靠的停止条件。

Q019 无限滚动页面为什么难抓？ ★★★☆☆

考察点

- 事件驱动加载
- XHR/fetch/cursor
- DOM 与数据源分离

推荐回答

无限滚动只是 UI 表现。背后往往是 scroll/IntersectionObserver 触发 fetch，服务端返回 JSON 和 cursor，再由前端渲染 DOM。

分析时先回答“数据从哪里来”，如果存在授权且稳定的结构化数据接口，应优先使用它；浏览器模拟滚动是成本更高的路径。

深度解析

浏览器方案还要定义停止条件：连续 N 次滚动没有新 item、next cursor 为空、达到业务上限等。

易错点

- 一上来就循环 scrollTo。
- 没有 item 去重，导致虚拟列表重复采集。

面试追问

1. 虚拟列表只保留可视节点时怎么办?
2. 如何判断滚动触发的是 API 还是 GraphQL?

一句话记忆

- 无限滚动不是“滚轮问题”，而是分页状态与数据源问题。

Q020 HTML parser 应该选择严格还是容错？ ★★☆☆☆

考察点

- 互联网 HTML 非规范性
- 容错修复

- 调试层次

推荐回答

真实网页存在未闭合标签、错误嵌套、非法字符和编码问题。严格 XML parser 容易直接失败，因此网页采集通常需要 HTML 容错 parser。

但容错 parser 会重构树，所以调试 selector 时要知道 parser 修复了什么；必要时同时保存原始字节与解析后的规范化 HTML。

深度解析

如果做法律/取证级归档，原始 response bytes 的保留比“修复后的 DOM”更重要。

易错点

- 用 XML parser 处理任意网页。
- 只存解析后的 HTML，丢失原始证据。

面试追问

1. lxml、html5lib 在容错行为上可能有什么差别?
2. 编码检测错误会如何污染 selector 和文本?

一句话记忆

- 网页解析优先容错，但原始响应必须可追溯。

第三章 Scrapy、异步与并发

理解 Scrapy 的事件驱动架构、调度、限速与 I/O/CPU 边界。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q021 画出 Scrapy 的架构。 ★★☆☆☆

考察点

- Engine
- Scheduler
- Downloader / Middleware / Spider / Pipeline

推荐回答

核心数据流是 Spider 产生 Request，Engine 交给 Scheduler；可调度请求再经 Downloader Middleware 到 Downloader；Response 回到 Engine，再经 Spider Middleware 交给 Spider；Spider 输出的新 Request 回到 Scheduler，Item 进入 Item Pipeline。

面试要说清楚 Engine 是协调者，Scheduler 管 frontier，Downloader 负责网络，Spider 负责解析和发现，Pipeline 负责数据后处理。

```
Spider → Engine → Scheduler
   ↑                ↓
Item ← Spider ← Response ← Downloader ← Middleware
```

深度解析

理解架构后，很多问题都有明确落点：代理/重试放 downloader middleware，字段校验放 pipeline，URL 发现逻辑放 spider。

易错点

- 把 Scheduler 当简单队列。
- 把所有逻辑都塞进 Spider。

面试追问

1. 重试应该放在哪一层？

2. URL 去重器和 Scheduler 的关系是什么?

一句话记忆

- Scrapy 的价值是把网络、调度、解析、数据处理拆成可组合的阶段。

Q022 Scrapy 为什么通常比 requests + ThreadPool 更适合高并发抓取?

★★★☆☆

考察点

- 事件驱动 I/O
- 等待时间利用
- 线程与协程成本

推荐回答

网页抓取往往是 I/O-bound：网络等待远大于 CPU 解析时间。Scrapy 基于 Twisted event loop，能在一个线程中维护大量 in-flight I/O，而不为每个请求都占用阻塞线程。

优势来自减少等待资源和调度成本，不是“异步在任何场景都更快”。当 OCR、PDF 解析、模型推理成为瓶颈时，应把 CPU/GPU 工作拆到独立 worker。

深度解析

线程池并非错误方案，小规模或同步依赖很多时也足够；系统设计回答要结合并发规模、库生态和 CPU/I/O 比例。

易错点

- 把 asyncio/Twisted 当成 CPU 并行。

面试追问

1. 什么时候 ThreadPool 反而更简单?
2. 如何避免 callback 中的 CPU work 阻塞 reactor?

一句话记忆

- 异步的核心收益是让等待不占用执行线程。

Q023 Reactor / Event Loop 是什么? ★★★☆☆

考察点

- 非阻塞 I/O
- 事件回调
- 并发与并行区别

推荐回答

Event loop 负责监听大量文件描述符或异步事件，某个 socket 就绪时再调度对应 callback/coroutine，而不是让线程阻塞等待。

因此 concurrency 是“多个任务进度交错”，parallelism 才是“多个 CPU 同时执行”。网络爬虫主要需要前者。

深度解析

任何在 event loop 上执行的长 CPU 任务都会阻塞所有其他连接，所以必须限制 callback 的执行时间。

易错点

- 认为单线程 event loop 没有并发。
- 在 callback 中做几秒 CPU 计算。

面试追问

1. epoll/kqueue 与 event loop 有什么关系?
2. Deferred/Future/Promise 在这里扮演什么角色?

一句话记忆

- Event loop 用事件驱动把等待时间“折叠”，但不能 magically 提供 CPU 并行。

Q024 Scrapy Scheduler 是做什么的? ★★★☆☆

考察点

- 请求排队
- 优先级
- 去重与持久化

推荐回答

Scheduler 接收 Spider/Engine 产生的 Request，结合去重器判断是否进入队列，并按优先级向 Engine 提供下一个请求。

生产场景还需要考虑持久化、恢复、不同域名优先级和延迟调度。单机 Scheduler 可以看作分布式 URL Frontier 的缩小版。

深度解析

如果想实现大规模 per-domain politeness，真正需要的是 host-aware frontier，而不只是一个全局优先队列。

易错点

- 把 Scheduler 与 Downloader 混为一谈。

面试追问

1. Scrapy 默认优先级队列对 BFS/DFS 有什么影响?
2. 如何把 Scheduler 状态持久化?

一句话记忆

- Scheduler 决定“下一步抓谁”，因此它承载了 crawler 的策略。

Q025 Downloader Middleware 和 Spider Middleware 有什么区别?

★★☆☆☆

考察点

- HTTP 层 hooks
- Spider 输入输出
- 职责边界

推荐回答

Downloader Middleware 位于 Engine 与 Downloader 之间，适合处理请求头、Cookie、重试、重定向、压缩、代理、请求指标等 HTTP 层行为。

Spider Middleware 位于 Engine 与 Spider 之间，更适合处理进入 Spider 的 Response/异常和 Spider 输出的 Request/Item。

深度解析

判断放哪层的方法是问：这个逻辑是否与具体页面解析无关？若是网络/传输关注点，通常靠近 downloader；若是 spider input/output 语义，靠近 spider middleware。

易错点

- 代理轮换、重试逻辑写进每个 parse 函数。

面试追问

1. robots middleware 属于哪一侧？为什么?

2. 异常重试和业务层重新调度有什么区别?

一句话记忆

- Middleware 的价值是把横切关注点从业务解析中抽离。

Q026 Item Pipeline 为什么不能和 Spider 混在一起? ★★☆☆☆

考察点

- 职责分离
- 数据验证与存储
- 可测试性

推荐回答

Spider 负责发现 URL、发请求、解析页面并产生结构化 Item；Pipeline 负责 normalize、validate、dedup、enrich 和 store。

这样 Spider 可以只关心网页语义，存储从 MySQL 切到 Kafka/Parquet 时无需重写抓取逻辑。

深度解析

数据 schema 演进也更容易：Pipeline 可以按版本统一补字段、转换单位和记录 validation failure。

易错点

- 在 Spider callback 里直接写数据库。

面试追问

1. Pipeline 中如何实现异步写入?
2. Item 去重应该按 URL 还是业务主键?

一句话记忆

- 发现与解析属于采集逻辑，规范化与落库属于数据处理逻辑。

Q027 Scrapy 如何控制并发? ★★☆☆☆

考察点

- 全局并发
- per-domain / per-IP 并发

- download delay

推荐回答

控制并发不能只看 CONCURRENT_REQUESTS。还要考虑每个 domain/IP 的并发、下载延迟和 AutoThrottle。

一个成熟配置围绕“目标站点可以承受多少并发”而不是“本机能开多少连接”。

深度解析

实际吞吐由连接池、DNS、服务器延迟、下载大小、callback CPU 时间和下游 backpressure 共同决定。

易错点

- 把并发设到很大后认为吞吐一定线性增长。

面试追问

1. per-domain 与 per-IP 有什么区别?
2. HTTP/2 单连接多流时并发计数怎么理解?

一句话记忆

- 并发控制是资源与礼貌策略，不是一个越大越好的数字。

Q028 AutoThrottle 是什么? ★★★☆☆

考察点

- 延迟反馈
- 目标并发
- 自适应速率控制

推荐回答

AutoThrottle 根据响应延迟动态调整每个 download slot 的 delay，让平均并发接近目标值。站点响应变慢时增加 delay，响应恢复时逐步减小。

它比固定 sleep 更接近反馈控制系统，但仍需设置最小/最大 delay、目标并发和错误处理边界。

深度解析

从控制论角度，它使用 latency 作为远端负载的代理信号。延迟并不总等于容量，因此不能仅靠一个指标判断。

易错点

- 把 AutoThrottle 当成反爬规避工具。
- 忽略 Retry-After 与 429。

面试追问

1. 如果 5xx 响应很快，AutoThrottle 会发生什么?
2. 怎样加入 error-rate feedback?

一句话记忆

- AutoThrottle 是反馈式限速，不是固定等待。

Q029 为什么不能在 Scrapy callback 里做长时间 CPU 运算? ★★★☆☆

考察点

- Event loop 阻塞
- I/O 与 CPU 解耦
- 工作池

推荐回答

Scrapy 的高吞吐依赖 reactor 快速处理事件。如果 callback 做 OCR、压缩、大模型推理或复杂 PDF 解析，event loop 会被占住，其他连接即使已经就绪也无法推进。

正确方式是将 CPU/GPU 密集任务交给线程/进程池或独立队列 Worker，并为输出建立 backpressure。

深度解析

需要同时监控 reactor lag、callback duration、CPU saturation 和 queue backlog，才能确认瓶颈是否已经从网络移动到计算。

易错点

- 仅看网络 QPS，不看 callback 延迟。

面试追问

1. 什么时候用线程池、什么时候用进程池?
2. 如何保证外部 worker 结果与原请求关联?

一句话记忆

- 事件循环适合快速调度 I/O，不适合承载长 CPU 任务。

Q030 Scrapy 如何实现 graceful shutdown? ★★★☆☆

考察点

- 信号处理
- frontier checkpoint
- 幂等与恢复

推荐回答

收到 SIGTERM 时，系统应停止接收新任务、给活跃请求合理时间完成、持久化 scheduler/frontier、flush pipeline 输出并安全退出。

否则部署、扩容、节点回收都会造成大量重复抓取或数据丢失。

深度解析

生产平台最好把任务状态设计成可租约/可恢复，worker 本身尽量无状态；这样即使强制退出，也能在 lease 到期后重投。

易错点

- 把进程退出当作异常场景，不设计恢复。

面试追问

1. 如何处理仍在写对象存储的大响应?
2. 如何确保 checkpoint 与 item commit 的顺序?

一句话记忆

- 可恢复性不是异常补丁，而是分布式 crawler 的基本能力。

第四章 JavaScript、SPA 与浏览器自动化

掌握动态渲染、网络事件、浏览器会话隔离和成本分层。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q031 requests 能拿到 HTML，为什么还是没有数据? ★☆☆☆☆

考察点

- CSR/SSR
- JS 动态加载
- 初始响应与最终 DOM

推荐回答

现代页面可能只返回一个空壳和脚本，数据在 JavaScript 执行后通过 fetch/XHR 获取，再写入 DOM。HTTP 客户端拿到的只是 server response，不会自动执行 JS。

因此第一步是确认数据是否已嵌在 HTML/JSON script 中，还是必须等待后续网络请求。

深度解析

不要把“没有 selector 结果”直接升级成浏览器。先检查 HTML source、script state、network waterfall，选择最便宜的可行数据路径。

易错点

- 遇到动态页就直接 Selenium。

面试追问

1. SSR、CSR、hydration 各是什么?
2. Next.js 页面数据可能藏在哪里?

一句话记忆

- 先定位数据生成路径，再决定是否需要浏览器。

Q032 SPA 爬虫应该先分析 DOM 还是 Network? ★★★☆☆

考察点

- 数据源定位
- API/GraphQL
- DOM 是呈现层

推荐回答

SPA 中 DOM 往往只是 API 数据经过前端组件渲染后的结果。调试优先看 Network，确认哪些 fetch/XHR/GraphQL 请求真正承载数据。

如果接口是公开且允许使用的，直接处理结构化响应通常比 DOM selector 更稳定、更低成本，也更容易做 schema validation。

深度解析

但不能把 DevTools 里看到的私有接口自动视为可任意调用；访问策略、认证和站点许可仍然要单独判断。

易错点

- “能看到 endpoint”就假设可以无限请求。

面试追问

1. GraphQL 单 endpoint 如何区分不同业务请求?
2. 前端做二次计算时怎样验证数据一致性?

一句话记忆

- SPA 抓取先找数据源，DOM 通常只是最后一公里。

Q033 Playwright 的 Browser、Context、Page 分别是什么? ★★☆☆☆

考察点

- 浏览器进程
- 会话隔离
- 页面/标签页

推荐回答

Browser 是浏览器进程级对象；BrowserContext 是轻量隔离会话，拥有自己的 cookies、storage 等状态；Page 是 Context 内的一张页面/标签。

大规模自动化通常复用 Browser，并创建多个 Context 作为会话隔离单元，比每个任务启动一个 Browser 更省资源。

深度解析

Context 仍需要生命周期管理：任务结束清理、最大页面数、内存阈值和 TTL，避免长期泄漏。

易错点

- 多账号共用 Context。
- 每个 URL 都冷启动 Chromium。

面试追问

1. Context 与 Chrome profile 有什么差异?
2. Context 崩溃或污染后如何回收?

一句话记忆

- Browser 管进程，Context 管身份状态，Page 管页面交互。

Q034 Selenium 和 Playwright 的主要区别? ★★☆☆☆

考察点

- WebDriver/BiDi
- 浏览器自动化能力
- 工程选型

推荐回答

Selenium 以 W3C WebDriver 为核心，生态成熟、跨语言跨浏览器广泛；现代 Selenium 也在使用 WebDriver BiDi 获取网络和浏览器事件。Playwright 更强调 context 隔离、auto-waiting、网络拦截与 tracing 的一体化体验。

选型应看现有技术栈、浏览器兼容、调试工具、并发规模、测试/采集需求，而不是简单宣称某框架“全面更强”。

深度解析

对于大规模采集，真正的成本通常来自浏览器渲染本身，框架差异只是其中一部分。

易错点

- 把 Selenium 描述成“只能点元素”。

面试追问

1. WebDriver BiDi 解决了传统 WebDriver 的什么限制?

2. 如何在两种框架之间做统一 Browser abstraction?

一句话记忆

- 框架选型是工程约束问题，不是品牌比较题。

Q035 为什么 sleep(5) 是浏览器自动化的坏味道? ★★☆☆☆

考察点

- 竞态条件
- 不必要等待
- 条件同步

推荐回答

固定 sleep 无法表达真正的就绪条件：页面 500ms 完成会白等，页面 8s 完成又会失败。应等待具体状态，例如 element visible、network response、URL change、某个 JS 状态或业务数据出现。

自动等待/显式等待的核心是把同步条件写进程序，而不是猜一个时间。

深度解析

等待条件也不能过于宽泛，例如“network idle”在长轮询页面可能永远不满足。应该围绕目标数据定义条件。

易错点

- 把 sleep 增大当成稳定性修复。

面试追问

1. 什么情况下固定小 sleep 仍可以接受?
2. 如何给等待条件设计超时和错误上下文?

一句话记忆

- 等待“条件”，不要等待“猜出来的秒数”。

Q036 什么是 explicit wait? ★★☆☆☆

考察点

- 条件轮询
- 超时

- 同步语义

推荐回答

Explicit wait 表示在最大 timeout 内反复检查一个条件，条件满足立即继续，否则超时失败。它把等待从“时间假设”变成“状态判断”。

常见条件包括元素存在/可见/可点击、某请求完成、文本出现、应用状态变化。

深度解析

高质量自动化会在 timeout 时输出可诊断上下文：当前 URL、页面截图、关键 DOM、网络失败和 console error。

易错点

- 条件只检查 element exists，但业务数据仍未加载。

面试追问

1. presence、visibility、clickable 有什么区别?
2. 如何等待特定 API response?

一句话记忆

- 显式等待的本质是状态机同步。

Q037 为什么不建议混用 implicit wait 和 explicit wait? ★★☆☆☆

考察点

- 嵌套超时
- 不可预测等待
- 调试困难

推荐回答

Implicit wait 会影响元素查找的默认等待，explicit wait 又在外层轮询条件；两者叠加后每次轮询可能被 implicit wait 拉长，最终 timeout 远超预期。

工程上应选择一种明确的同步策略，通常优先 explicit/auto-wait，并让 timeout 可局部配置。

深度解析

复杂系统还要区分 navigation timeout、element timeout、API timeout，避免一个全局值掩盖不同失败类型。

易错点

- 把所有 timeout 都设成 60 秒。

面试追问

1. 如何定位一个“本应 5 秒却等了 30 秒”的 case?
2. 页面导航超时和元素等待超时应如何分层?

一句话记忆

- 超时必须可预测，否则失败恢复和容量估算都会失真。

Q038 什么是 Chrome DevTools Protocol（CDP）？ ★★★☆☆

考察点

- 浏览器调试协议
- Network/Runtime/DOM
- 低层事件与控制

推荐回答

CDP 是 Chromium 暴露的调试协议，DevTools 本身也使用它。通过不同 domain 可以观察网络请求、运行 JavaScript、读取 DOM、性能指标等。

它提供比传统“找元素并点击”更底层的浏览器能力，因此常用于网络监听、性能分析、页面状态采集。

深度解析

CDP 是 Chromium 特定协议；跨浏览器标准化方向是 WebDriver BiDi。回答时应区分“能力强”与“标准通用”。

易错点

- 把 CDP 当成 HTTP 抓包工具的别名。

面试追问

1. CDP 与 WebDriver BiDi 的定位差异?
2. Network.requestWillBeSent 与 responseReceived 能做什么?

一句话记忆

- CDP 是浏览器内部调试事件和控制能力的协议化接口。

Q039 Playwright 如何观察 Network Request? ★★☆☆☆

考察点

- request/response/requestfinished
- network failure 与 HTTP error
- 事件关联

推荐回答

Page/BrowserContext 可以监听 request、response、requestfinished、requestfailed 等事件。要特别注意：HTTP 404/503 仍然是成功获得了 HTTP response，并不等于 network failure。

采集系统可通过请求 URL、resource type、response status、content-type 过滤目标 API，并把 network event 与页面业务事件关联。

深度解析

网络监听是调试和数据来源分析工具，但不要用它绕过站点明确设置的访问挑战。

易错点

- 把 404 当 requestfailed。

面试追问

1. 如何拿到 response body 并和请求参数关联?
2. Service Worker 会如何影响网络观察?

一句话记忆

- HTTP 错误码是应用层响应；network failure 是请求没有正常得到响应。

Q040 Browser Crawler 最大的性能问题是什么? ★★★☆☆

考察点

- 进程/渲染成本
- JS 与资源加载
- 分层 fast path

推荐回答

浏览器路径不仅下载 HTML，还要启动/维护浏览器进程、执行 JS、解析 CSS、布局渲染、加载图片字体等资源，CPU 和内存成本远高于普通 HTTP fetch。

因此成熟架构采用 HTTP fast path：能直接解析就不进浏览器；只有确实需要渲染的页面进入 Browser Queue/Pool。

深度解析

还可以在合法业务需求下阻止无关大资源、限制页面生命周期、复用 BrowserContext，但不能以破坏站点安全机制为目标。

易错点

- 所有 URL 一律 Chromium。

面试追问

1. 如何估算 Browser Pool 容量?
2. 什么指标决定页面需要升级到 browser path?

一句话记忆

- 浏览器是昂贵执行环境，应当是分层采集系统中的慢路径。

第五章 分布式爬虫

把单机脚本升级为可扩展、可恢复、可治理的 URL Frontier 与工作流系统。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q041 单机爬虫怎么演化成分布式爬虫？ ★★★★☆

考察点

- 状态外置
- URL Frontier
- Fetcher/Parser/Storage 解耦

推荐回答

单机脚本里 scheduler、seen set、解析和存储都在一个进程。分布式演化的第一步是把状态外置：URL Frontier、去重索引、任务 lease、结果存储都不依赖某个 worker 内存。

随后把 Fetcher 与 Parser 分开，通过队列形成可扩缩的流水线，并给每一阶段建立幂等、重试、backpressure 和可观测性。

深度解析

分布式的难点不是“多起几个进程”，而是共享状态、一致性、故障恢复和跨 worker 的 per-host politeness。

易错点

- 把 requests + Redis 直接等同于分布式爬虫。

面试追问

1. 哪些状态必须全局一致，哪些可以分区？
2. Fetcher 与 Parser 为什么适合拆开？

一句话记忆

- 分布式化的本质是把状态和执行解耦，而不是复制单机脚本。

Q042 分布式 crawler 最难的问题是什么? ★★★★☆

考察点

- Frontier
- 去重/分片
- 故障与礼貌策略

推荐回答

下载本身通常不是最难。真正困难的是 URL frontier、全局去重、分区、延迟调度、recrawl freshness、故障恢复、backpressure 和跨节点一致的 per-host 限速。

这些问题本质上把 crawler 变成一个带外部约束的分布式调度系统。

深度解析

面试时可把问题分成三个平面：Data Plane（fetch/parse）、Control Plane（policy/rate/priority）、State Plane（frontier/dedup/history）。

易错点

- 只讨论“加机器”和“加 Kafka”。

面试追问

1. 如何保证同一 host 在多个 worker 上仍只使用一个全局预算?
2. Frontier 的优先级更新如何做?

一句话记忆

- 大规模 crawler 的核心竞争力在调度与状态管理，而不是 HTTP 库。

Q043 URL Frontier 是什么? ★★★☆☆

考察点

- 待抓 URL 集合
- 优先级
- 延迟与策略

推荐回答

URL Frontier 是已发现但尚未完成抓取的 URL 集合及其调度状态。它不是简单 FIFO，而应综合重要性、freshness、domain budget、深度、失败历史和抓取成本。

可以抽象 Priority(u)=a*Importance+b*Freshness-c*Cost-d*Risk，并同时满足 next_allowed_fetch(host) 等硬约束。

深度解析

搜索引擎 crawler 还需要 recrawl queue 与 discovery queue 协同，既发现新网页又保持重要网页新鲜。

易错点

- 用一个 Redis list 表示所有策略。

面试追问

1. 优先队列如何支持 future timestamp?
2. 如何防止一个 host 的大量 URL 淹没 frontier?

一句话记忆

- Frontier 决定抓取价值与时机，是 crawler 的“大脑”。

Q044 URL 如何分片到不同 Worker? ★★★☆☆

考察点

- hash(host)
- 局部性
- hot partition

推荐回答

常见方法是按 host/domain 做一致性哈希或 hash(host)%N，把同一 host 尽量放到同一调度分区。这样 robots、连接池、host rate limit 和域级状态更容易局部化。

但简单哈希会遇到热门域倾斜，因此需要虚拟分区、二级队列或 host scheduler 与 worker pool 解耦。

深度解析

分片键不是纯数据分布问题，还决定你能否方便地实施目标站点级约束。

易错点

- 直接随机把每个 URL 分发给 worker，导致每台都单独限速。

面试追问

1. 一致性哈希扩容时为什么更友好?
2. host 分片与 registrable domain 分片如何取舍?

一句话记忆

- 分片应同时考虑负载均衡和策略局部性。

Q045 如何解决热门域名导致的 Partition Skew? ★★★★☆

考察点

- 热点识别
- 虚拟分区
- 全局 host budget

推荐回答

当一个域占据大量 URL 时，单纯按 host 哈希会让一个 partition 爆满。可以让 host scheduler 保持全局访问预算，再把该 host 的可执行任务分发到多个虚拟分区/worker。

关键是“执行可并行、预算不能被放大”。即使 10 个 worker 都能抓同一站点，也要共享一个 host token bucket。

深度解析

对热点 host 还可做 adaptive batching、连接复用和更精细的 priority，以提高单位请求价值。

易错点

- 扩 10 台机器就让 host QPS 变 10 倍。

面试追问

1. 全局 token bucket 放哪里?
2. 如何避免 rate-limit service 成为单点瓶颈?

一句话记忆

- 热点域需要拆执行，不拆访问预算。

Q046 Kafka 能不能直接当 URL Frontier? ★★★★☆

考察点

- 持久日志
- 缺少 priority/delay
- 组合架构

推荐回答

Kafka 擅长高吞吐持久日志、分区和 consumer group，但 crawler frontier 还需要优先级、future scheduling、per-host politeness、URL 去重和 recrawl 更新。

因此 Kafka 可以承载任务流/事件流，但通常还需要独立 scheduler state 或多 topic/timing layer 才能实现真正 frontier。

深度解析

如果业务只是离线批量抓一组固定 URL，Kafka 足够；如果是 Web-scale discovery crawler，单纯 Kafka 不够表达策略。

易错点

- 看到队列就回答 Kafka。

面试追问

1. Kafka partition 内怎么做优先级?
2. 延迟任务如何实现?

一句话记忆

- Kafka 是可靠流组件，不天然等于“带策略的 URL Frontier”。

Q047 Redis 在分布式 crawler 中适合做什么? ★★★☆☆

考察点

- 轻量状态
- 计数器/锁/优先队列
- 内存成本

推荐回答

Redis 适合保存小规模队列、租约、分布式计数器、短期 rate-limit state、缓存和 priority set。

但几十亿 URL 的长期 seen set、完整 history 或大 payload 放在 Redis 会非常昂贵，应考虑 Bloom Filter、RocksDB、对象存储或分区 KV。

深度解析

设计回答要给出状态生命周期：热状态放 Redis，长期状态放持久 KV/LSM，原始内容放对象存储。

易错点

- 把 Redis 当无限内存数据库。

面试追问

1. ZSET 做延迟队列的复杂度是什么?
2. Redis 故障恢复如何影响去重状态?

一句话记忆

- Redis 适合热状态和协调，不适合无边界长期数据。

Q048 如何保证 Worker Crash 后任务不丢? ★★★★☆

考察点

- lease/visibility timeout
- ACK
- 重投与幂等

推荐回答

任务被 worker 获取时不是立刻从系统永久删除，而是获得一个 lease/visibility timeout；处理成功后 ACK，worker 崩溃则 lease 到期，任务重新可见。

这种机制天然是 at-least-once，所以存储和解析必须幂等。

深度解析

对于长任务，需要 heartbeat/lease extension；否则仍在处理的任务可能被重复分配。

易错点

- “pop 出队列”后立刻丢状态。

面试追问

1. Worker 卡死但没退出怎么办?
2. 如何处理 poison task 无限重投?

一句话记忆

- 可靠队列的核心是租约，而不是“拿走即删除”。

Q049 为什么 Exactly-once Crawling 几乎没有必要? ★★★★☆

考察点

- 分布式不确定性
- at-least-once + idempotency
- 成本收益

推荐回答

“请求已经成功、结果也写了，但 worker 在 ACK 前崩溃”时，调度器无法原子地知道远端网页请求和本地事务的全部状态。强行追求 end-to-end exactly-once 成本极高。

更实用的做法是 at-least-once fetch、幂等解析/存储、URL 和内容去重，把少量重复执行变成可接受。

深度解析

Exactly-once 更多是业务效果语义，而非“网络上只发送一次”。

易错点

- 把 Kafka exactly-once 误认为整个网络采集链路 exactly-once。

面试追问

1. 如何定义一条网页抓取任务的幂等键?
2. 同 URL 内容随时间变化时还能用 URL 做幂等键吗?

一句话记忆

- 大多数 crawler 追求“结果不重复”，而不是“网络请求物理上只发生一次”。

Q050 什么是 Backpressure? ★★★☆☆

考察点

- 生产/消费速率不匹配
- 队列膨胀
- 反馈控制

推荐回答

若 Fetcher 100k pages/s，而 Parser 只能 20k，队列会持续增长，最终导致内存、磁盘、延迟和重试风暴。Backpressure 就是让下游容量反向影响上游生产速率。

可根据 queue lag、inflight、storage latency、CPU saturation 自动减少 fetch concurrency 或暂停低优先级任务。

深度解析

成熟系统不追求每个 stage 的局部最大吞吐，而追求整个 pipeline 的稳定吞吐。

易错点

- 只扩 fetcher，不观察下游。

面试追问

1. Queue lag 与 queue length 哪个更重要?
2. 如何避免 backpressure 导致所有 host 都被一起减速?

一句话记忆

- 流水线的速度由最慢环节决定；上游必须感知下游容量。

第六章 去重、调度、存储与数据质量

决定抓取结果能否长期复用、增量更新和稳定进入下游。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q051 URL 去重最简单怎么做? ★★☆☆☆

考察点

- Hash Set
- 内存估算
- 大规模替代

推荐回答

最简单是 seen=set(url_hash)。查询平均 O(1)，但规模上升后哈希表元数据成本巨大。10 亿条记录即使每条只有 8 字节 hash，也已经约 8GB，真实哈希表通常远高于此。

因此大规模系统会采用分区 KV、LSM/RocksDB、Bloom Filter 或分层 dedup：热集合精确，冷集合近似。

深度解析

去重键通常应使用 canonical URL 或其稳定 hash，而不是原始字符串。

易错点

- 只按 Python set 的理想 8 字节估算内存。

面试追问

1. Bloom Filter 假阳性可接受到什么程度?
2. 如何做跨天持久化 seen set?

一句话记忆

- URL 去重首先是容量问题，其次才是算法题。

Q052 Bloom Filter 为什么适合 URL 去重? ★★★☆☆

考察点

- 概率集合
- 空间效率
- 假阳性

推荐回答

Bloom Filter 用多个 hash 映射到 bit array，空间效率高，插入和查询都是 O(k)。标准实现没有传统假阴性，但允许假阳性：没见过的 URL 可能被误判为已见。

对于大规模发现型 crawler，少抓极少量页面可能比为精确集合付出巨大内存更可接受。

深度解析

是否可接受假阳性取决于业务：搜索引擎 coverage 可能希望更低 FPR；合规归档或财务采集则可能要求精确去重。

易错点

- 说 Bloom Filter “100% 准确”。

面试追问

1. Counting Bloom Filter 能解决什么?
2. Bloom Filter 满载后为什么 FPR 会恶化?

一句话记忆

- Bloom Filter 用可控漏抓概率换数量级更低的内存。

Q053 Bloom Filter 假阳性率怎么估? ★★★☆☆

考察点

- m/n/k
- 最优 hash 数
- 容量规划

推荐回答

经典公式 p=(1-exp(-k*n/m))^k，其中 m 是 bit 数、n 是元素数、k 是 hash 数。最优 k 约为 (m/n)*ln2。

面试中不要只背公式，要说明容量估算流程：先给目标 n 与可接受 p，求每元素 bit 数，再估总内存，并留增长余量。

深度解析

当插入量超出设计 n 时，FPR 会快速上升，因此需要分代 Bloom、扩容或周期重建。

易错点

- 只给公式，不做量级估算。

面试追问

1. 如果 URL 规模从 10 亿增长到 20 亿怎么办?
2. hash 函数数量增加为什么不一定更好?

一句话记忆

- Bloom Filter 的工程价值来自“可预先计算的空间-误判权衡”。

Q054 URL Canonicalization 为什么困难? ★★★☆☆

考察点

- 规范化与语义
- query 参数
- 错误合并风险

推荐回答

scheme/host 大小写、默认端口、fragment、追踪参数、路径规范化都可能产生等价 URL，但 query 参数排序、重复参数、大小写和空值是否等价则依赖站点语义。

因此 canonicalization 应分成通用安全规则 + 域级规则，不要“一刀切删除 query”。

深度解析

还可以使用 rel=canonical、重定向和内容 hash 作为后验信号，逐步学习站点 URL 等价关系。

易错点

- 全量 sort query parameters。
- 删除所有 utm 之外的参数但没有域级测试。

面试追问

1. fragment 在哪些页面可能影响内容?
2. session id 参数如何处理?

一句话记忆

- URL 规范化的最大风险不是漏合并，而是把不同资源错误合并。

Q055 URL 去重和内容去重有什么区别? ★★☆☆☆

考察点

- 地址重复
- 内容重复
- 两阶段

推荐回答

URL dedup 防止同一或等价 URL 重复进入 frontier；content dedup 在抓取后判断不同 URL 是否承载相同或近似内容。

典型系统先 URL dedup 降低网络成本，再对正文做 exact hash、MinHash/SimHash 等内容去重，减少下游存储和训练/索引冗余。

深度解析

两层去重不能相互替代：镜像站不同 URL 需要内容去重；同 URL 随时间更新又不能永久内容合并。

易错点

- 只做 URL 去重就认为不存在重复内容。

面试追问

1. 内容去重应对原始 HTML 还是正文做?
2. 同文章不同语言翻译算重复吗?

一句话记忆

- URL 去重解决“别重复请求”，内容去重解决“别重复保存同一信息”。

Q056 网页内容如何做近重复检测? ★★★☆☆

考察点

- Shingling
- Jaccard
- MinHash + LSH

推荐回答

先将规范化正文转为词/字符 n-gram 集合，再用 Jaccard 衡量集合重叠。直接两两比较是 O(N^2)，因此使用 MinHash 近似 Jaccard，并通过 LSH 找候选相似文档。

它适合镜像页面、轻度模板变化和转载，但不擅长深度语义改写。

深度解析

规范化过强会误合并，过弱会漏检。网页场景通常要先去 boilerplate，否则公共导航会人为提高相似度。

易错点

- 把 LSH 的转折点当硬阈值。

面试追问

1. n-gram 取大取小分别有什么影响?
2. 如何选择重复簇中的代表文档?

一句话记忆

- 近重复检测的核心是“低成本候选生成 + 可解释相似度”。

Q057 什么时候使用 SimHash? ★★★☆☆

考察点

- 局部敏感哈希
- Hamming Distance
- 与 MinHash 对比

推荐回答

SimHash 把文档特征映射成固定长度 bit fingerprint，相似文档的指纹 Hamming distance 较小。它适合快速近重复 fingerprint 和 bit-index 检索。

MinHash 更直接对应 Jaccard set similarity；SimHash 更像把高维加权特征压成超平面符号。

深度解析

选哪种取决于特征和检索基础设施，不应机械说“新闻用 SimHash、网页用 MinHash”。

易错点

- 把 SimHash 当密码学 hash。

面试追问

1. 64-bit SimHash 在超大规模下有哪些碰撞风险?
2. 如何做 Hamming distance <=3 的索引?

一句话记忆

- SimHash 的价值是相似文档产生“近的 bit 指纹”，而非完全相等。

Q058 数据应该存 MySQL、MongoDB 还是对象存储? ★★★☆☆

考察点

- 冷热分层
- 原始/规范化/服务层
- 数据生命周期

推荐回答

不存在一种存储适合全部阶段。Raw HTML/response body 适合对象存储；结构化分析结果适合 Parquet/Data Lake；需要事务和主键查询的业务实体适合 SQL；搜索服务适合倒排索引。

设计时先问访问模式、更新模式、保留周期、数据量与成本，而不是先选品牌。

深度解析

一个常见三层是 Raw Zone（不可变原始数据）→ Curated Zone（规范化列式数据）→ Serving Layer（数据库/搜索）。

易错点

- 把所有 HTML 塞 MySQL TEXT。
- 只存最终字段，无法重新处理。

面试追问

1. Parquet 为什么适合分析层?
2. 对象存储上的小文件问题怎么解决?

一句话记忆

- 存储选型由数据生命周期和访问模式决定。

Q059 为什么原始 HTML 最好保留? ★★☆☆☆

考察点

- 可重放
- parser 版本升级
- 审计/provenance

推荐回答

Selector 和抽取规则会变化。如果只保存最终字段，发现 parser 错误后必须重新访问站点；保存原始 response 后，可以离线 replay 新 parser。

Raw 数据还支持审计：知道某条结构化事实来自哪个 URL、什么时间、什么响应。

深度解析

建议保存 raw body、fetch timestamp、headers 摘要、status、content hash、crawler/parser version；必要时保存压缩后的原始字节。

易错点

- 只保存 screenshot 不保存可解析响应。

面试追问

1. 如何控制 raw retention 成本?
2. robots/删除请求发生后 raw 数据生命周期如何治理?

一句话记忆

- 原始数据是可重放和可审计的底座。

Q060 成熟 crawler 的数据模型应该有哪些 Metadata? ★★★☆☆

考察点

- provenance
- 版本
- 抓取与解析上下文

推荐回答

至少记录 url、canonical_url、fetch_timestamp、status_code、content_type、response_size、content_hash、etag、last_modified、crawler_version、parser_version 和 source。

动态页面还应记录 render_mode、browser/version、关键 context；结构化 item 再记录 extraction method、validation status、entity key。

深度解析

元数据看似增加存储，但能显著降低排障、回滚、重新解析和数据删除成本。

易错点

- 只保存 URL 和正文。

面试追问

1. 如何让 provenance 从 item 反查到 raw response?
2. 版本字段放 commit hash 还是 semantic version?

一句话记忆

- 没有 provenance 的抓取数据，很难成为可靠数据资产。

第七章 反爬检测原理

从防守视角理解现代 Bot Detection 的多信号、风险评分与误报成本。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q061 服务端如何判断一个请求是不是 Bot? ★★★☆☆

考察点

- 多信号融合
- 风险评分
- 规则 + ML

推荐回答

现代 Bot Detection 通常综合网络、HTTP、TLS、浏览器能力、Cookie/session 历史、行为节奏、IP/ASN reputation 等信号，而不是单看 User-Agent。

系统把特征送入规则和模型，输出 risk/bot score，再由策略层决定 allow、throttle、challenge 或 block。

深度解析

防御设计的真正难点是 false positive：合法 API client、搜索 crawler、无障碍工具和企业自动化也可能表现出“非人类”流量。

易错点

- 认为某一个特征可以稳定判定所有 bot。

面试追问

1. 哪些特征适合在 edge 10ms 内计算?
2. 如何给已验证的合法 crawler 建 allowlist?

一句话记忆

- 现代反爬是多信号风险建模，而不是字符串匹配。

Q062 为什么只改 User-Agent 不能解决 Bot Detection? ★★☆☆☆

考察点

- 单特征易伪造
- 跨层一致性
- 会话历史

推荐回答

User-Agent 只是一个自报字符串。服务端还可看到 TLS/HTTP 协议特征、Cookie 连续性、JS 能力、访问行为、IP reputation 等。

如果自报浏览器身份与其他层特征不一致，反而可能提高风险分。

深度解析

从防御视角，应避免把单个可轻易伪造字段设为强判据，而应看跨信号一致性。

易错点

- 把 UA 轮换当作完整策略。

面试追问

1. Client Hints 与 UA 有什么关系?
2. 为什么跨层一致性比单一字段更有价值?

一句话记忆

- 身份是多层行为的一致性，不是一个 Header。

Q063 什么是浏览器指纹? ★★★☆☆

考察点

- 环境特征组合
- 稳定性/熵
- 隐私与误报

推荐回答

浏览器会暴露屏幕、locale、timezone、字体、图形能力、API feature support 等环境属性。防欺诈/反自动化系统可能组合这些信号形成设备或会话 fingerprint。

指纹不是绝对身份：浏览器升级、隐私保护、企业设备同质化都会造成变化或碰撞。

深度解析

好的防御系统把 fingerprint 当概率信号，并与账户、行为和历史关联，而不是用它作唯一封禁条件。

易错点

- 把指纹当“永久唯一设备 ID”。

面试追问

1. 哪些特征稳定但隐私风险高?
2. 如何评估 fingerprint collision?

一句话记忆

- 指纹是概率性环境画像，不是身份证。

Q064 TLS Fingerprint 是什么? ★★★☆☆

考察点

- ClientHello
- cipher/extensions
- 网络栈特征

推荐回答

TLS 握手中的 ClientHello 包含 cipher suites、extensions、supported groups、versions 等组合，不同客户端库和浏览器版本可能呈现不同模式。

防守方可以把它作为“客户端实现特征”之一，与声明的 HTTP/browser identity 做一致性分析。

深度解析

TLS 指纹会随版本、GREASE、代理和中间层变化，因此不能当绝对证据。

易错点

- 认为同一浏览器永远只有一个 TLS 指纹。

面试追问

1. JA3/JA4 的思想是什么?
2. HTTP/2 SETTINGS fingerprint 与 TLS fingerprint 有何区别?

一句话记忆

- TLS 指纹描述的是客户端网络栈行为，而不是应用层自报身份。

Q065 IP Reputation 是怎么工作的? ★★★☆☆

考察点

- ASN/网络类型
- 历史行为
- 时间衰减

推荐回答

防御系统会把 IP 与 ASN、数据中心/住宅/移动网络类型、历史滥用、错误率、请求量、地理位置和跨站情报等关联，形成随时间变化的 reputation。

它应是一个动态风险信号，而非永久黑名单，因为 NAT、云资源复用和 IP 重新分配都会改变主体。

深度解析

面试中要强调单 IP 限制对 NAT 用户会产生误伤，也不能阻止多 IP 分布式自动化。

易错点

- 认为数据中心 IP 一定恶意。

面试追问

1. 如何给 reputation 做时间衰减?
2. NAT 环境下如何降低误伤?

一句话记忆

- IP 只是网络入口，reputation 必须结合时间和上下文。

Q066 为什么数据中心 IP 与普通用户网络的风险画像可能不同? ★★☆☆☆

考察点

- 先验分布
- ASN
- 不能单独定罪

推荐回答

云数据中心网络天然承载服务器、爬虫、API 和攻击流量，因此从统计分布看可能与家庭/移动网络不同。

但它只是 prior；很多合法企业服务、监控、搜索引擎也运行在数据中心，不能凭 ASN 就直接封禁。

深度解析

成熟策略会把网络类型作为模型特征，并结合账户、行为、请求目的和已知合法身份。

易错点

- 把“风险较高”说成“必然恶意”。

面试追问

1. 如何识别已验证搜索引擎 crawler?
2. 企业客户固定出口 IP 应如何处理?

一句话记忆

- 网络来源是先验特征，不是最终判决。

Q067 行为检测主要看什么? ★★★☆☆

考察点

- 时间序列
- 页面转移
- 会话一致性

推荐回答

常见行为信号包括请求间隔、burstiness、页面转移图、资源加载模式、会话持续时间、交互事件和错误恢复模式。

系统关注的是行为序列是否符合某类合法流量分布，而不是某一次点击是否“像人”。

深度解析

行为模型需要防概念漂移：产品改版、用户行为变化和移动端流量变化都会改变正常分布。

易错点

- 用单次鼠标事件作为强证据。

面试追问

1. 如何构造 session-level feature?
2. 如何避免模型把高频合法 API 客户误判成 bot?

一句话记忆

- 行为检测看序列和上下文，而不是单点动作。

Q068 为什么固定频率访问容易形成异常特征？ ★★☆☆☆

考察点

- 周期性
- burstiness
- 单特征不足

推荐回答

机械的精确周期会产生低方差时间序列，在某些流量类型中可能与真实用户分布不同，因此可作为风险信号之一。

但监控系统、合法 crawler 和定时 API client 同样可能周期访问，所以不能只凭 timing 封禁。

深度解析

防御系统更可靠的方式是把 timing 与 endpoint、身份、资源加载、失败模式和 reputation 组合。

易错点

- 认为“随机延时”就能解决所有检测。

面试追问

1. 如何用 inter-arrival time 建模?
2. 周期性 API client 如何声明合法身份?

一句话记忆

- 时间模式可提示自动化，但必须和其他信号结合。

Q069 CAPTCHA 在 Bot Management 体系中处于什么位置？ ★★★☆☆

考察点

- 挑战成本
- 风险分层
- 用户体验

推荐回答

CAPTCHA/Challenge 通常是中间策略，而不是所有请求的第一层。更常见流程是先用廉价 edge signals 计算风险，低风险放行，中风险 challenge，高置信恶意再 block。

挑战本身有用户体验、无障碍、延迟和转化成本，所以策略优化需要把误报成本计算进去。

深度解析

服务端可以使用 managed challenge、JS detection 或无交互验证等不同成本的手段逐级升级。

易错点

- 对所有匿名用户都上 CAPTCHA。

面试追问

1. 怎样用挑战结果反哺风险模型?
2. Challenge solve rate 可以说明什么?

一句话记忆

- Challenge 是不确定区间的验证手段，不是默认入口。

Q070 为什么“出现 CAPTCHA 就自动破解”不是成熟 crawler 的设计?

★★★☆☆

考察点

- 访问边界
- 业务授权
- 稳定替代路径

推荐回答

当站点返回 CAPTCHA/Challenge，成熟采集系统应把它视为访问策略状态：暂停、降低频率、使用站点提供的 API/数据出口、或通过授权渠道解决，而不是把系统目标设成自动绕过。

这同时是可靠性问题：对抗式绕过往往脆弱、成本高、随站点升级频繁失效。

深度解析

企业平台应记录 challenge rate、受影响域名和授权状态，避免 worker 在同一边界上无限重试。

易错点

- 无限重试 challenge。
- 把绕过挑战当成爬虫能力 KPI。

面试追问

1. 如果 challenge 突然从 1% 升到 50%，如何排查?
2. 有哪些非对抗性的替代数据获取方式?

一句话记忆

- 遇到明确挑战应转向策略与授权处理，而不是对抗升级。

第八章 服务端防爬体系

限流、WAF、Bot Management、挑战策略与模型评估。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q071 如何设计最简单的 Rate Limiter? ★★☆☆☆

考察点

- 计数器
- 时间窗口
- 分布式状态

推荐回答

最简单固定窗口：对 key（IP、account、API key 等）在每个窗口内计数，超过阈值拒绝或降速。实现简单、存储低。

问题是窗口边界突发：一个客户端可以在前一秒用完旧窗口额度，下一秒又立刻用完新窗口额度。

深度解析

在分布式网关中，计数器要考虑原子递增、TTL、地域一致性与故障时的 fail-open/fail-close 策略。

易错点

- 只写伪代码，不谈窗口边界和分布式一致性。

面试追问

1. Redis INCR+EXPIRE 有什么竞态?
2. 限流服务故障时应该放行还是拒绝?

一句话记忆

- 固定窗口简单但有边界 burst，适合从 MVP 开始。

Q072 Sliding Window 有什么优势? ★★★☆☆

考察点

- 平滑速率
- 边界 burst
- 实现成本

推荐回答

Sliding Window 按当前时刻向前看一个窗口，避免固定窗口边界在极短时间内允许双倍流量，更接近真实“最近 W 秒”的速率。

可用精确日志（ZSET 时间戳）或近似滑动计数器实现。精确方案状态更多，近似方案更省资源。

深度解析

高 QPS 网关通常不会为每个请求永久保存时间戳，需要做桶化或近似。

易错点

- 只讲优点，不谈 state cost。

面试追问

1. Sliding log 与 sliding counter 区别?
2. 高基数 key 怎么控制内存?

一句话记忆

- Sliding Window 用更多状态换更平滑准确的限速。

Q073 Token Bucket 原理是什么? ★★★☆☆

考察点

- 平均速率
- burst capacity
- 补充速率

推荐回答

桶容量 B，token 以速率 r 补充，请求消耗 token。长期平均速率受 r 限制，但允许利用积累 token 进行最大 B 的短期突发。

它非常适合 API 限流，因为同时表达“平均预算”和“允许合理 burst”。

深度解析

分布式实现要统一时钟或使用基于 elapsed time 的惰性补充算法，避免频繁定时任务。

易错点

- 把 B 与 r 混为一个参数。

面试追问

1. 如何用 Redis Lua 原子实现?
2. 多地域 active-active 下 token 如何分配?

一句话记忆

- Token Bucket = 平均速率 r + 突发额度 B。

Q074 Leaky Bucket 和 Token Bucket 区别? ★★☆☆☆

考察点

- 流量整形
- burst
- 队列模型

推荐回答

Token Bucket 允许积累额度后突发；Leaky Bucket 更强调以稳定速率“漏出”流量，输入突发被排队或丢弃。

因此前者常用于“允许合理 burst 的配额”，后者常用于“平滑输出”的 traffic shaping。

深度解析

实际系统可以把两者组合：入口 token bucket 控制预算，下游 leaky/queue 控制平滑发送。

易错点

- 认为两者只是名字不同。

面试追问

1. 如果 Search 后端怕突发，哪个更合适?
2. 队列满时 Leaky Bucket 应如何处理?

一句话记忆

- Token Bucket 允许积累额度，Leaky Bucket 强调平滑输出。

Q075 只按 IP Rate Limit 有什么问题? ★★★☆☆

考察点

- NAT 误伤
- 多 IP 绕过
- 多维身份

推荐回答

一个 IP 后面可能有企业、学校或运营商 NAT 的大量真实用户；反过来自动化流量也可能分布在很多 IP。

因此限流常使用 IP、account、session、API key、endpoint、ASN 等多个维度，并设置 global overload protection。

深度解析

设计时需明确匿名用户与登录用户的不同 key 层次，以及多 key 的组合规则。

易错点

- 一条 IP 规则覆盖所有业务。

面试追问

1. 同一请求同时命中 IP 和 account 限额怎么办?
2. IPv6 前缀该如何作为 key?

一句话记忆

- IP 只是一个限流维度，不是稳定用户身份。

Q076 如何保护一个昂贵 Search API? ★★★★☆

考察点

- 成本感知限流
- 端点配额
- 过载保护

推荐回答

Search 往往触发数据库、检索、排序和模型调用，单位请求成本远高于静态资源。应按 endpoint 设置独立 budget，再叠加 account/user/global 限额。

还可以按 query complexity、结果页大小、模型推理成本实施 weighted token bucket，而不是所有请求都消耗 1 个额度。

深度解析

过载时优先降级昂贵功能、减少结果数、关闭次要 rerank，而不是等数据库完全崩溃。

易错点

- 所有 endpoint 一个统一 QPS。

面试追问

1. 如何设计 weighted cost?
2. 缓存命中的 Search 请求是否应消耗同样配额?

一句话记忆

- 限流应反映业务成本，而不只是请求数量。

Q077 WAF 与 Bot Management 有什么区别? ★★★☆☆

考察点

- 攻击规则
- 自动化分类
- 协同策略

推荐回答

WAF 更传统地处理已知攻击模式、路径/参数/头规则、IP/ASN、速率限制等；Bot Management 更关注自动化识别、行为、浏览器信号、reputation 与 ML score。

实际系统两者常协作：Bot score 可以成为 WAF/Policy Engine 的一个条件，WAF 也提供阻断与挑战执行能力。

深度解析

边界越来越模糊，面试应讲职责而不是品牌功能列表。

易错点

- 认为 WAF 只防 SQL 注入。

面试追问

1. Bot score 如何进入 WAF policy?
2. 什么时候规则比模型更可靠?

一句话记忆

- WAF 偏请求安全策略，Bot Management 偏自动化风险识别，两者在策略层汇合。

Q078 Challenge、Block、Throttle 应该如何选择? ★★★☆☆

考察点

- 风险分层
- 误报成本
- 渐进式响应

推荐回答

低风险直接 allow；轻度异常可 throttle；不确定但风险较高可 challenge；只有高置信恶意且业务容忍误报时才 block。

策略不是纯分类问题，而是决策问题：需要比较误报、漏报、用户体验、基础设施成本和攻击损失。

深度解析

可以对不同 endpoint 设置不同阈值：登录、支付、搜索、静态资源的风险容忍度不同。

易错点

- 一个 bot score 阈值全站 block。

面试追问

1. 如何做 shadow policy?
2. 挑战后通过的用户如何临时降低风险分?

一句话记忆

- 风险越不确定，越应该使用可逆、低损失的策略。

Q079 如何评价一个反爬模型? ★★★★☆

考察点

- Precision/Recall
- FPR/FNR
- 成本加权与分群

推荐回答

流量通常高度不平衡，因此 accuracy 几乎没有意义。应看 precision、recall、FPR、FNR、PR-AUC，并把 false positive 与 false negative 的业务成本显式写进目标函数。

还要按网络类型、地区、设备、登录状态、客户类型分群评估，防止总体指标掩盖某一群体误伤。

深度解析

线上评估通常需要 shadow、A/B、人工回查、挑战结果和申诉数据，而不是只看离线标签。

易错点

- 只报 99.9% accuracy。

面试追问

1. 如何给没有真值标签的线上流量评估?
2. 为什么 PR-AUC 常比 ROC-AUC 更有用?

一句话记忆

- 反爬模型评价的核心是业务成本与误报，而非总体准确率。

Q080 如何设计反爬系统的 Feature Pipeline? ★★★★☆

考察点

- Edge 特征
- 在线聚合
- 低延迟评分

推荐回答

Request 在 edge 提取网络/TLS/HTTP 等廉价信号，再读取 session、account、IP/ASN、历史速率等状态特征，送入规则与模型，得到 risk score，最后由 Policy Engine 执行 allow/throttle/challenge/block。

高成本特征不应全部同步计算；可采用热特征缓存、异步 enrichment 和分层模型，在 5-20ms 的边缘延迟预算内完成核心决策。

深度解析

还要设计 feature version、model version、decision log，保证事后能解释“为什么被挑战”。

易错点

- 离线训练特征与线上特征实现不一致。

面试追问

1. 如何避免 feature store 成为延迟瓶颈?
2. 特征缺失时模型应如何降级?

一句话记忆

- 反爬 Feature Pipeline 是实时特征工程 + 决策系统，而不是单一模型调用。

第九章 稳定性、可观测性与数据质量

生产爬虫最重要的不是“能跑”，而是“能发现自己已经错了”。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q081 Crawler 最重要的指标是什么？ ★★★☆☆

考察点

- 业务指标
- 系统指标
- 数据质量指标

推荐回答

requests/sec 只是局部吞吐。成熟 crawler 更关注 UsefulItems/sec、fetch success、parse success、items/page、字段缺失率、duplicate ratio、freshness lag、queue lag、p95/p99 latency 和字节成本。

指标应分层：网络层、解析层、数据层、调度层和业务层。

深度解析

只有把“抓到了多少有用且新鲜的数据”作为最终目标，优化才不会被局部 QPS 带偏。

易错点

- 只看 HTTP 成功率和 QPS。

面试追问

1. 什么是 freshness SLA?
2. 如何定义一条 item 的 useful?

一句话记忆

- 爬虫 KPI 应从“请求数量”升级为“有用数据产出与新鲜度”。

Q082 为什么 HTTP 200 不能代表抓取成功? ★★☆☆☆

考察点

- 传输成功 vs 业务成功
- 软错误页
- 语义验证

推荐回答

200 只说明服务器成功返回一个 HTTP 实体。内容可能是登录页、错误模板、challenge 页面、空列表或降级页。

因此 fetch success 后仍需做 content-type、模板识别、正文长度、业务字段和页面类型验证。

深度解析

可对常见软错误页计算 fingerprint，监控 200 响应中异常模板的比例。

易错点

- status==200 就计成功。

面试追问

1. 如何识别 200 登录页?
2. 页面返回“暂无数据”时是成功还是失败?

一句话记忆

- Transport success 不等于 extraction success。

Q083 如何发现网站改版? ★★★☆☆

考察点

- 字段分布漂移
- DOM signature
- 报警

推荐回答

对每个 page type 监控 parse success、required field missing rate、items/page、文本长度分布和 selector fallback 使用率。网站改版往往先表现为统计分布突变。

还可以保存 DOM/HTML signature 或少量代表页面做定期 regression diff。

深度解析

报警应结合变化幅度与持续时间，避免站点短暂实验或空数据时频繁误报。

易错点

- 只等业务方发现数据坏了。

面试追问

1. DOM signature 如何设计得对内容变化不敏感?
2. 如何区分站点 A/B 实验与永久改版?

一句话记忆

- 改版检测本质是数据与结构分布漂移检测。

Q084 什么叫 Silent Data Corruption? ★★★☆☆

考察点

- 无异常但数据错误
- 字段空值
- 比 Crash 更危险

推荐回答

程序继续运行、HTTP 也正常，但 selector 失效、单位解析错、字段映射错，导致大量错误数据进入下游，这就是 silent corruption。

它比 crash 更危险，因为监控看起来“健康”，错误会在数据仓库、推荐、风控或训练中扩散。

深度解析

防线包括 schema validation、统计基线、交叉字段检查、canary、抽样审核和 raw replay。

易错点

- 用 try/except 吞掉所有异常并返回空字段。

面试追问

1. 如何定义数据质量 SLO?
2. 哪些异常应该 fail closed?

一句话记忆

- 最危险的爬虫故障不是停机，而是悄悄生产错误数据。

Q085 如何做 Crawler Regression Test? ★★★☆☆

考察点

- HTML fixtures
- golden output
- parser diff

推荐回答

保存覆盖典型模板和边界情况的 HTML fixtures，Parser 修改后离线运行新旧版本，对结构化输出、字段值、数量和异常进行差异比较。

测试应固定 raw input，避免回归测试本身依赖实时网站。

深度解析

对选择器易变站点，可把真实线上样本定期匿名化后加入 fixture corpus，形成逐步扩充的回归集。

易错点

- 测试直接访问线上页面，结果不可重复。

面试追问

1. 如何处理时间字段这种天然变化值?
2. golden file 更新流程怎么防止“把 bug 更新成基线”？

一句话记忆

- Parser 应像普通软件一样有固定输入、确定输出和回归集。

Q086 如何做 Canary Crawling? ★★★☆☆

考察点

- 灰度发布
- 新旧对比
- 自动回滚

推荐回答

新 crawler/parser 版本先处理 1% 或少量域名，比较 success rate、字段分布、items/page、latency、error code 和成本。确认无退化后逐步扩大到 10%、50%、100%。

高风险变化应支持自动 rollback：一旦关键指标跌破阈值，立即恢复旧 parser。

深度解析

Canary 不只是基础设施灰度，也适合 selector、浏览器版本、抽取算法和调度策略。

易错点

- 直接全量发布 parser。

面试追问

1. 新旧 parser 输出不完全一致时如何判定优劣?
2. Canary 样本如何覆盖长尾页面类型?

一句话记忆

- 灰度把“线上试错”变成受控实验。

Q087 Retry 应如何设计? ★★★☆☆

考察点

- 错误分类
- 指数退避 + jitter
- Retry-After

推荐回答

Retry 先分类错误：网络 timeout、连接重置、部分 5xx 可能重试；永久 404、无权限、无效 URL 通常不重试。

采用指数退避 delay=min(max, base*2^n) 并加入 jitter，避免大量 worker 同时失败后同步重试形成 thundering herd。尊重 Retry-After。

深度解析

还要设置总重试预算，而不仅是单任务次数；大规模故障时应 circuit break，阻止所有任务继续轰击同一故障域。

易错点

- 所有非 200 都重试三次。

面试追问

1. 429 和 503 的 backoff 有何不同?
2. 如何设计 domain-level circuit breaker?

一句话记忆

- Retry 是受预算的恢复策略，不是无限循环。

Q088 哪些错误不应该反复 Retry? ★★☆☆☆

考察点

- 永久错误
- 策略拒绝
- 毒任务

推荐回答

明确的永久 404/410、非法 URL、unsupported content、策略拒绝、无效 schema 或逻辑上不可恢复的数据通常不应重复重试。

这类任务应记录原因并进入终态或 DLQ；只有状态可能改变的错误才值得 schedule retry。

深度解析

错误分类本身也要版本化，因为某些 403 可能代表临时限流或永久权限拒绝，需要结合响应语义。

易错点

- 把 4xx 一律永不重试，或一律重试。

面试追问

1. DNS NXDOMAIN 是否应该重试?
2. 解析错误是修 parser 还是重抓页面?

一句话记忆

- 重试价值取决于错误未来是否可能自行恢复。

Q089 Dead Letter Queue 是什么? ★★☆☆☆

考察点

- 失败隔离
- 人工分析
- 可重放

推荐回答

任务超过重试预算后进入 DLQ，而不是无限占据主队列。DLQ 保存 task id、输入、失败原因、最后错误、版本和次数，支持后续聚类、修复和 replay。

DLQ 是稳定系统的“故障隔离区”，避免少量 poison tasks 拖垮整体。

深度解析

DLQ 也要治理，不能只“扔进去不看”；应有按原因统计、SLA 和清理/重放流程。

易错点

- DLQ 没有输入上下文，无法重放。

面试追问

1. Parser 修复后如何批量 replay?
2. DLQ 堆积到多少需要报警?

一句话记忆

- DLQ 让不可恢复错误退出主流程，同时保留可诊断与可重放能力。

Q090 如何定位“爬虫今天突然变慢”？ ★★★★☆

考察点

- 延迟分解
- 队列/CPU/存储
- 数据驱动排障

推荐回答

先把端到端耗时拆成 DNS、connect、TLS、TTFB、download、parse、store，再看 queue lag、CPU、memory、GC、连接池和 Browser Pool。

如果 TTFB 变慢，可能是目标站或网络；如果 parse 时间上涨，可能是页面体积/算法；如果 store latency 上涨，则 fetcher 即使正常也会被 backpressure。

深度解析

排障必须先定位层次，再判断是否是反爬或外部策略变化，不能凭感觉修改请求行为。

易错点

- 看到 QPS 下降就判断“对方反爬升级”。

面试追问

1. 如何用 tracing 关联单个 URL 的各阶段耗时?
2. p99 上升但 p50 正常说明什么?

一句话记忆

- 把“慢”拆成可测阶段，才能从猜测变成定位。

第十章 高级系统设计

用容量估算、约束建模和分层架构回答 10^8 到 10^9 URL 级系统设计题。

维度	本章答题要求
协议/语义	能解释 Why，不只会调 API
工程/实现	能指出状态、失败和资源边界
面试/表达	先结论，再原理，再 trade-off，再追问

Q091 设计一个每天抓取 1 亿网页的系统。 ★★★★★

考察点

- 容量估算
- 分布式 Frontier
- 限速/容错/存储

推荐回答

先做量级：100,000,000 / 86,400 ≈ 1,157 pages/s 平均吞吐。若按 3 倍峰值设计，抓取层约 3,500 pages/s；再根据平均响应大小估带宽和对象存储写入。

架构：Seed → URL Normalize/Dedup → Distributed Frontier → Domain-aware Scheduler → Fetcher Pool → Raw Object Storage → Parser/Validation → Content Dedup/Entity Resolution → Data Lake/Serving。

控制平面独立管理 robots、per-host token bucket、retry/circuit breaker、优先级和 recrawl；数据平面专注 fetch/parse。

```
Seed → Normalize/Dedup → Frontier → Host Scheduler → Fetchers
                                                        ↓
                                                  Raw Object Store
                                                        ↓
                                              Parser → Validate → Lake
```

深度解析

容量还要乘失败重试、动态页面比例和平均页面大小。例如 500KB/page，则 1 亿页原始体量约 50TB/day（未压缩、未扣压缩差异）。

如果 5% 页面走浏览器，Browser Pool 是单独容量模型，不能按普通 fetcher QPS 估算。

易错点

- 只说“Kafka + Redis + Scrapy 集群”没有任何容量数字。

- 忽略 per-domain 限速导致全局吞吐与站点约束冲突。

面试追问

1. 如何把 1157 pages/s 转成机器数?
2. 存储 30 天 raw data 大约多少?
3. 如何保证重要站点 freshness?

一句话记忆

- 系统设计题先算量级，再画数据面/控制面/状态面。

Q092 设计一个搜索引擎 Web Crawler。 ★★★★★

考察点

- coverage
- freshness
- importance/crawl traps/spam

推荐回答

搜索引擎 crawler 同时优化新网页发现和已知网页新鲜度。URL 来自 seed、页面链接、sitemap、历史库和 recrawl。

调度可抽象 Priority(u)=α·Importance+β·Freshness-γ·Cost-δ·Risk，并受到 host politeness、robots、容量和重复预算约束。

还要处理 canonical、crawl trap、spam、duplicate、redirect graph 和 change-rate learning。

深度解析

“抓全网”不是现实目标，必须在有限带宽下最大化索引价值。可用 PageRank/站点质量、历史变化概率和查询需求共同定义 crawl value。

易错点

- 只讲 BFS/DFS。

面试追问

1. 新 URL 与 recrawl URL 如何争夺预算?
2. 如何估页面 change probability?
3. 如何防 infinite calendar trap?

一句话记忆

- 搜索 crawler 是受资源约束的 coverage × freshness 优化问题。

Q093 如何设计 Recrawl Scheduler? ★★★★★

考察点

- 变化率估计
- 条件请求
- SLA

推荐回答

不同 URL 变化频率差异巨大。保存 last_fetch、last_change、ETag、Last-Modified、content hash 和历史变化间隔，估计 P(change | URL, Δt)。

长期不变页面逐步延长间隔，频繁变化页面缩短间隔；同时受业务 freshness SLA 和站点访问预算限制。

深度解析

可以用 hazard rate/指数移动平均近似变化率；重要页面即使变化率低，也可能因 SLA 要求维持最小抓取频率。

易错点

- 固定“每天全量重抓”。

面试追问

1. 如何处理突发新闻页?
2. 内容 hash 变化但正文没变（广告变了）怎么办?
3. 条件请求如何降低成本?

一句话记忆

- Recrawl 是变化概率、页面价值和访问预算的联合调度。

Q094 如何检测 Crawl Trap? ★★★★★

考察点

- URL 空间爆炸
- 参数基数
- 内容重复

推荐回答

典型 trap 包括无限日历、session 参数、搜索组合、排序过滤组合和循环链接。检测信号包括 URL pattern explosion、路径深度、参数 cardinality、重复 token、host 新 URL 速率和内容相似度。

策略包括 host URL budget、pattern quota、参数白/黑名单、深度限制、近重复反馈和人工规则。

深度解析

高阶做法是把 URL template 聚类，例如 /calendar/{yyyy}/{mm}，对 template 级别限制扩张速度。

易错点

- 只按 crawl depth 限制，错过正常深层页面。

面试追问

1. 如何自动学习 URL template?
2. 内容重复率高时何时停止整个 pattern?

一句话记忆

- Crawl trap 是“无穷 URL 空间 + 低新增信息量”，要在 pattern 层控制。

Q095 如何抓取 1000 个域名，同时保证每个域名不超过自己的 QPS?

★★★★★

考察点

- per-host queue
- token bucket
- eligible-host scheduling

推荐回答

每个 host 有自己的 queue 和 token bucket（r_h, B_h），Global Scheduler 只从当前有 token 且 next_allowed_fetch 已到的 host 中选择任务，再分发给 Fetcher。

这样 worker 数量可以横向扩展，但 host 总 QPS 不随 worker 数量放大。

深度解析

为了避免扫描 1000 个 host，可以用最小堆按 next_eligible_time 管理 host；热门 host 仍保持一个共享 budget。

易错点

- 每个 worker 自己 sleep，导致全局预算乘以 worker 数。

面试追问

1. host scheduler 单点如何扩展?
2. 域名权重不同如何实现 weighted fairness?

一句话记忆

- 把访问预算绑定到 host，而不是绑定到 worker。

Q096 设计一个“HTTP + Browser”混合爬虫。 ★★★★★

考察点

- Fast path / slow path
- 内容能力检测
- Browser Queue

推荐回答

所有 URL 先走 HTTP Fetch。Classifier/规则判断初始 HTML 是否含足够数据：若可解析则进入 fast parser；否则进入 Browser Queue，由 Browser Pool 渲染后再解析。

需要防止重复升级：记录 render_required/page_type，缓存静态判断；Browser Pool 采用 Context 隔离，设置超时、TTL、资源限制和健康检查。

深度解析

关键 KPI 是 Browser Path Ratio，因为 browser 成本通常高一个数量级以上。可以通过 SSR state、JSON-LD、授权结构化接口等方式降低该比例。

易错点

- 把 browser 当默认路径。

面试追问

1. 如何训练 render-required classifier?
2. HTTP 与 Browser 结果不一致时保留哪个?
3. Browser queue 积压如何 backpressure?

一句话记忆

- 混合架构的目标是把昂贵浏览器严格限制在真正需要的页面。

Q097 Browser Pool 应该怎么设计? ★★★★★

考察点

- 进程复用
- Context 隔离
- 资源回收

推荐回答

少量 Browser process 长期运行，每个任务创建/复用受控 BrowserContext；Page 完成后关闭，Context 按任务或 TTL 回收。

Pool 需要最大并发、单 Context 页面数、内存阈值、任务超时、崩溃重启、健康检查和队列。

深度解析

容量估算不要只看“每台能开多少 page”，而要按平均 CPU、RSS、页面耗时和峰值并发计算，并保留崩溃/GC 余量。

易错点

- 一个 Browser 永不重启导致内存泄漏。
- 不同身份共用状态。

面试追问

1. 如何处理 browser crash 导致的在途任务?
2. 什么时候按 Context 回收，什么时候整 Browser 回收?

一句话记忆

- Browser Pool = 进程复用 + 会话隔离 + 有界生命周期。

Q098 设计一个企业级反爬系统。 ★★★★★

考察点

- Edge 低延迟
- Feature/Model/Policy 分层
- 误报治理

推荐回答

数据面：Request → Edge feature extraction → session/account/reputation state → rules + model → risk score → policy engine。执行动作包括 allow、throttle、challenge、block。

状态面：维护速率、IP/ASN reputation、账户历史、设备/session 摘要；控制面：模型版本、规则发布、allowlist、阈值、shadow/A-B 和 rollback。

可观测性必须记录 feature version、score、decision reason、challenge outcome 和申诉结果。

深度解析

延迟预算通常非常紧，因此 expensive feature 不能同步全算；采用 edge cache、分层模型、异步 enrichment。

模型与规则的最终目标是最小化业务成本：攻击损失 + 误伤损失 + challenge UX 成本。

易错点

- 把系统画成“一个 ML 模型 → block”。

面试追问

1. 如何做模型 shadow deployment?
2. 跨地域 reputation state 怎么同步?
3. 合法 crawler 如何认证?

一句话记忆

- 企业反爬不是一个分类器，而是实时风险决策平台。

Q099 Crawler 成功率从 99% 突然降到 30%，怎么排查? ★★★★★

考察点

- 分层排障
- 指标证据
- 不要先入为主

推荐回答

按层排查：①基础设施 DNS/network/CPU/queue/DB；② HTTP 状态分布、redirect、response length、latency；③页面模板/HTML 变化；④ Parser selector/schema；⑤ Browser JS error/timeout/crash；⑥ robots、rate-limit、授权/访问策略变化。

每一步都通过指标和样本证据缩小范围，而不是一上来修改 Header、代理或自动化行为。

深度解析

先对“成功率”的定义做分解：fetch success、parse success、item validation success。三者下降的根因完全不同。

易错点

- 把所有 403 都归因于反爬升级。

面试追问

1. 如果只有一个站点下降怎么判断?
2. 如果所有站点同时下降最可能看哪里?
3. 怎样做自动 root-cause hints?

一句话记忆

- 故障排查先分层定义成功，再用指标定位，不要凭经验猜。

Q100 终极题：设计一个十亿级 URL、可增量更新、合规、稳定且不压垮目标网站的爬虫平台。 ★★★★★

考察点

- 全局架构
- 控制/状态/数据三平面
- 容量、合规、可恢复

推荐回答

整体链路：Seed Management → URL Normalize → Global Dedup → Distributed Frontier → Policy/robots evaluation → Domain-aware Scheduler → HTTP Fetcher / Browser Pool → Raw Data Lake → Parsing/Validation → Content Dedup/Entity Merge → Serving Storage。

状态面保存 URL history、robots、ETag/Last-Modified、last_fetch/last_change、host budget、failure score、dedup index；控制面管理策略、版本、限流、授权、删除请求和审计。

抓取层采用 at-least-once + idempotent store，错误走有预算的 retry/circuit breaker/DLQ；backpressure 从 storage/parser 反向控制 scheduler。

```
Seed → Normalize → Dedup → Frontier → Policy → Host Scheduler
                                      ├→ HTTP Fetch
                                      └→ Browser Pool
                                              ↓
                                        Raw Data Lake
                                              ↓
                                  Parse → Validate → Dedup → Serve
```

深度解析

容量上先估 10^9 URL 的 frontier 与 history 状态：即使每 URL 仅几十字节也是几十 GB 到数百 GB，必须分片持久化；raw body 更是 PB 级生命周期问题。

增量更新利用 ETag/Last-Modified/content hash 与 change-rate learning；调度目标可写成最大化 Σ Value(u)·Freshness(u)，约束每个 domain 的 RequestRate ≤ Budget。

合规上至少要区分 robots 协商、授权/许可、登录权限、数据保留/删除与审计，不把“技术可访问”当作“业务可采集”。

易错点

- 系统设计只画组件，不写约束和状态。
- 全局扩容时忘记 host QPS 不能跟着倍增。
- 没有 raw provenance 和版本，无法审计/重放。

面试追问

1. Frontier 如何按 host 与时间双重调度?
2. 如何做 10^9 URL 去重与扩容?
3. 如何控制 PB 级 raw retention?
4. 如何把 freshness SLA 映射成 recrawl priority?

一句话记忆

- Web-scale crawler 是一个受站点预算、数据价值、freshness 和成本共同约束的分布式调度系统。

附录 A：7 天冲刺复习路线

天	主题	完成标准
Day 1	HTTP + URL + Robots + Cache	能白板解释请求链路、304、幂等、robots。
Day 2	DOM + 抽取 + 数据质量	能设计稳定 selector、正文抽取与 schema validation。
Day 3	Scrapy + Async	能画架构、解释 event loop、并发与 backpressure。
Day 4	JS + Browser	能判断何时需要浏览器、设计 Browser Pool。
Day 5	分布式 + 去重 + 存储	能设计 Frontier、Bloom、lease、raw/curated/serving。
Day 6	反爬防护	能设计 rate limit、risk score、WAF/Bot Management。
Day 7	系统设计 10 题	每题 15 分钟：估算→架构→状态→故障→SLO。

附录 B：高级面试回答模板

1. 先给结论：一句话说明你会怎么做。
2. 给原理：解释为什么这样做，指出关键协议/算法/系统约束。
3. 给工程实现：数据结构、组件、状态、指标和失败处理。
4. 给 Trade-off：成本、精度、延迟、误报、覆盖率之间如何权衡。
5. 主动指出边界：什么情况下该方案失效，如何演进。

系统设计题黄金顺序

- 1. 明确目标与约束：规模、freshness、站点预算、数据准确性。
- 2. 做量级估算：pages/s、带宽、存储、状态规模、浏览器比例。
- 3. 画数据流：Seed → Frontier → Fetch → Raw → Parse → Validate → Store。
- 4. 补控制平面：robots、per-host budget、priority、retry/circuit breaker。
- 5. 补状态平面：dedup、history、ETag、last_fetch、leases。

- 6. 最后讲 SLO、监控、故障恢复和成本。

附录 C：核心公式与速查

```
Average throughput = pages_per_day / 86400
Bloom FPR: p = (1 - exp(-k*n/m))^k
Bloom optimal k ≈ (m/n) * ln(2)
Jaccard(A,B) = |A ∩ B| / |A ∪ B|
Retry delay_n = min(max_delay, base * 2^n) + jitter
Token Bucket: average rate <= r, burst <= B
Crawler objective: maximize Σ Value(u) * Freshness(u)
Constraint: RequestRate(host_i) <= Budget_i
```

参考资料

RFC 9309 - Robots Exclusion Protocol: https://www.rfc-editor.org/rfc/rfc9309.html

RFC 9110 - HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110.html

RFC 9111 - HTTP Caching: https://www.rfc-editor.org/rfc/rfc9111.html

Scrapy Architecture: https://docs.scrapy.org/en/latest/topics/architecture.html

Scrapy Downloader Middleware: https://docs.scrapy.org/en/latest/topics/downloader-middleware.html

Scrapy AutoThrottle: https://docs.scrapy.org/en/latest/topics/autothrottle.html

Playwright BrowserContext: https://playwright.dev/docs/api/class-browsercontext

Playwright Network: https://playwright.dev/docs/network

Selenium WebDriver: https://www.selenium.dev/documentation/webdriver/

Selenium Waits: https://www.selenium.dev/documentation/webdriver/waits/

Chrome DevTools Protocol: https://chromedevtools.github.io/devtools-protocol/

Cloudflare Bot Score Concepts: https://developers.cloudflare.com/bots/concepts/bot-score/

Cloudflare Challenges: https://developers.cloudflare.com/cloudflare-challenges/

注：本文引用资料主要用于协议、框架和防护概念的技术校准。具体产品版本、网站条款和法律要求可能变化，实际工程应以目标系统当前文档与授权为准。